

day02- 文本切分和向量存储

一、文本切分

大型语言模型（LLM）有两个硬性限制：

1. **上下文窗口限制**：模型单次能处理的文本长度是有限的（如 4K、8K、128K tokens）。
2. **检索精度问题**：在 RAG 架构中，如果把整本书作为一个整体存入向量数据库，检索时只能返回整本书，模型无法精准定位到具体的段落。

文本分割器（Text Splitters） 的作用就是将大型文档拆解为**语义连贯、大小合适**的小块（Chunks），使其既能塞进模型的上下文窗口，又能被向量数据库精准检索。

```
1  # 使用 pip 安装
2  pip install -U langchain-text-splitters
3
4  # 或使用 uv 安装
5  uv add langchain-text-splitters
```

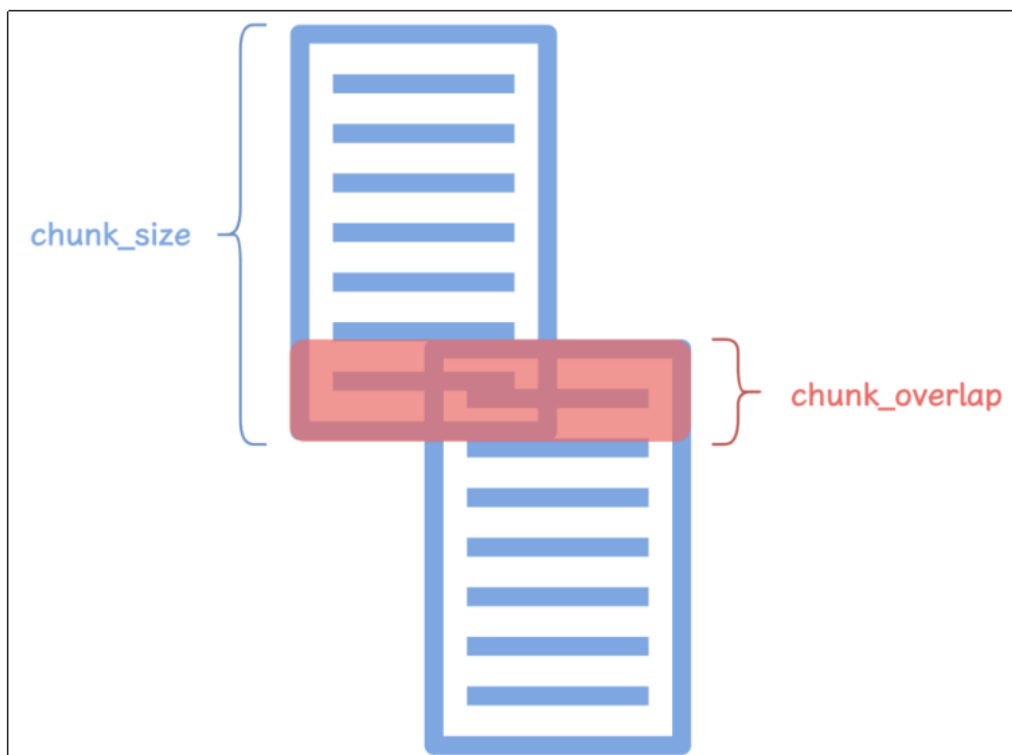
1.1 基础介绍

langchain-text-splitters 中提供了多种文档分块方式，包括：

- CharacterTextSplitter（根据分隔符按字符拆分文档）
- RecursiveCharacterTextSplitter（递归使用多个分隔符按字符拆分文档）
- TokenTextSplitter（使用模型分词器将文本拆分为 token）
- SentenceTransformersTokenTextSplitter（使用句子模型分词器将文本拆分为 token）

等。

文本分块时常用的两个参数 `chunk_size` 和 `chunk_overlap`，分别定义了块的大小和两个块之间重叠部分的大小。



策略一：基于文本结构（递归分割）

原理：自然语言具有天然的层级结构（段落 > 句子 > 单词）。递归分割器会尝试在最高层级（如段落）保持文本完整；如果段落超长，则降级到句子级别；如果句子还长，再降级到单词级别。

优势：最大程度保留了自然语言的流畅度和语义连贯性，不会生硬地从词语中间切断。

💡 对于绝大多数通用场景（如 PDF、TXT 文档的 RAG），**请始终将 `RecursiveCharacterTextSplitter` 作为您的默认首选**。它开箱即用，效果极佳。只有在特定需求下才考虑其他策略。

```
1  from langchain_text_splitters import RecursiveCharacterTextSplitter
2
3  # 文本分块
4  text_splitter = RecursiveCharacterTextSplitter(
5      # 优雅降级策略：优先按双换行切，再按单换行，最后按标点和空格
6      separators=["\n\n", "\n", "。", "!", "?", ";", ".", "!", "?", ";", " "], #
7      # 分隔符列表
8      chunk_size=400, # 每个块的最大长度
9      chunk_overlap=40, # 每个块重叠的长度
10 )
11 texts = text_splitter.split_documents(documents)
12 #pieces = splitter.split_text(body)
```

降级切分顺序：

1. "\n\n" - 双换行（段落边界）
2. "\n" - 单换行
3. "。！？；" - 中文句子结束标点
4. ".!?!;" - 英文句子结束标点
5. " " - 空格（最后手段）

策略二：基于长度（精准控制）

原理：严格按照设定的长度阈值进行“一刀切”式的分割。主要分为**按字符（Character）**和**按 Token**两种。

应用场景：

- 当您使用的 LLM 对 Token 数量有极其严格的限制和计费标准时。
- 当处理非自然语言文本（如随机字符串、日志数据）时。

代码示例（基于 Token 分割）： 使用 `tiktoken`（OpenAI 的分词器）可以确保切分后的块绝对不会超出模型的 Token 限制。

```
1  from langchain_text_splitters import CharacterTextSplitter
2
3  # 使用 OpenAI 的 cl100k_base 编码计算 Token
4  text_splitter = CharacterTextSplitter.from_tiktoken_encoder(
5      encoding_name="cl100k_base",
6      chunk_size=100, # 这里的 100 指的是 100 个 Token，而不是字符
7      chunk_overlap=0
8  )
9
10 texts = text_splitter.split_text(document)
```

策略三：基于文档固有结构（语义感知）

原理：许多文档自带标记语言结构（如 Markdown 的 `#` 标题、HTML 的 `<div>` 标签、JSON 的 `{}` 括号）。这种策略会**尊重文档的语法树**，确保一个完整的代码块或一个完整的章节不会被从中间腰斩。

应用场景：

- **Markdown：**按 H1, H2, H3 标题切分，非常适合技术文档和笔记。
- **HTML：**按 DOM 节点切分，适合网页抓取后的清洗。
- **JSON：**按对象或数组元素切分，适合结构化 API 数据。
- **Code：**按函数、类或逻辑块切分，适合代码库的 RAG（如 GitHub Copilot 的底层逻辑）。

可用工具列表：

- `MarkdownHeaderTextSplitter` (保留标题元数据)
- `RecursiveJsonSplitter`
- `Language` (支持 Python, JS, C++ 等几十种编程语言的分割)
- `HTMLHeaderTextSplitter`

补充：工业级 RAG 系统切片核心架构

在构建面向具体业务场景的知识库问答大脑时，文档切片（Chunking）往往决定了整体架构的智商下限。初期的误区在于，认为只需把完整的 PDF 或 Markdown 暴力灌入向量数据库即可。但在工业级应用中，精细的文本切分是不可或缺的基石。

五个核心维度，深度解析切片策略在工程中的绝对必要性：

1. 检索精度维度

目的：对抗“语义稀释”效应

- **物理限制：**Embedding 模型（如 BGE-M3）通常会将输入文本映射为一个固定维度的稠密向量（如 768 维或 1024 维）。
- **语义稀释灾难：**如果直接把包含复杂技术细节的万字长文映射为单一向量，文章中具体的方法论、代码片段或特定规则的特征，会被庞杂的宏观语义完全淹没（即所谓的“大海捞针”困境）。
- **切片收益：**通过切分，我们将大文本打碎成聚焦的“语义原子”。当用户提问非常具体的细节时，拥有单一、高浓度语义的 Chunk 能够在向量空间中被极高概率地精准命中。

2. 经济与性能维度

目的：突破 Token 与算力成本控制

- **上下文瓶颈：**尽管当前的主流大模型支持 128K 甚至更长的上下文，但将海量原文档全部塞入 Prompt，会导致首字生成时间（TTFT）呈指数级上升，产生严重的交互延迟。
- **财务黑洞：**商业大模型的 API 严格按 Token 计费。工业级系统每天面对海量并发，每次都携带冗长且无关的背景上下文，是不可接受的资源浪费。
- **切片收益：**切片后，RAG 系统只需挑选召回得分最高的 Top-K（如 3-5 个最相关的分片）喂给大模型。这不仅将系统响应压缩至毫秒级，更将大模型推理的 Token 成本降低了几个数量级。

3. 生成质量维度

目的：提升信噪比与物理抑制“幻觉”

- **迷失在中间 (Lost in the Middle)：**当 LLM 处理超长上下文时，对文本开头和结尾的注意力较高，但极易遗忘或混淆中间地带的键信息。
- **信息干扰：**如果在询问“如何取消订单”时，检索系统带回了包含“创建订单”、“退货退款”的整章内容，冗余信息极易诱导模型张冠李戴，产生事实性幻觉。
- **切片收益：**精准的结构化切片物理隔绝了不相关的干扰信息。投喂给 LLM 的上下文变成了高纯度“干货”，极大提升了最终答案的逻辑严密性和准确率。

4. 数据治理与溯源维度

目的：元数据挂载的前提

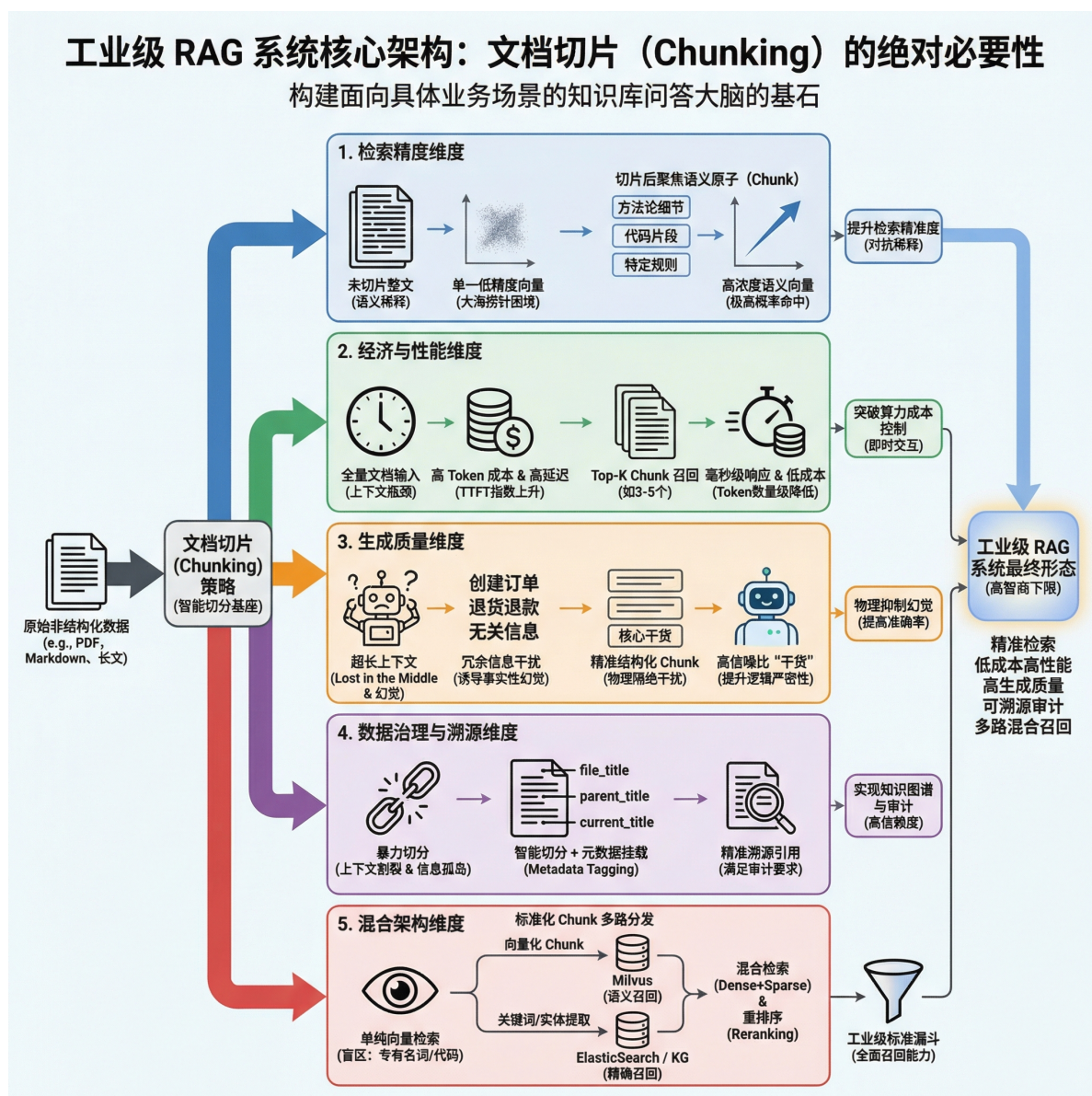
- **信息孤岛陷阱**：单纯的按字数（Token）暴力切分会导致上下文割裂，这也是通常为什么我们需要引入基于 Markdown 标题树（Heading Tree）的智能切分。
- **切片收益**：切片不仅是“切”，更是“装”。切分过程让我们能够为每一个 Chunk 打上极其详细的元数据标签（如 `file_title`, `parent_title`, `current_title`）。这不仅使得知识图谱的构建成为可能，也让系统能够在生成答案时附带极其精准的文件溯源引用，满足高信赖度场景的审计要求。

5. 混合架构维度

目的：多路召回（Hybrid Search）的要求

- **单纯向量检索的盲区**：向量检索擅长处理“模糊语义泛化”，但在遭遇专有名词、商品编号、等特定代码片段的精确匹配时，效果极差。
- **切片收益**：切片后，每一个标准化的 Chunk 可以兵分两路：一路被向量化存入 Milvus 提供语义召回，另一路被提取关键词甚至实体存入知识图谱或 Elasticsearch 提供精确召回。没有细粒度的切分基座，混合检索（Dense + Sparse）加重排序（Reranking）的工业级标准漏斗便无从谈起。

全方位图解分析



1.2 Markdown 切分原理

如果说普通的文本分割器（如按字符或 Token 切割）是“盲人摸象”，那么

`MarkdownHeaderTextSplitter` 就是“带着目录看书”。它能够理解 Markdown 的层级结构，将文本按标题进行语义分组，并将标题信息作为**元数据（Metadata）**完美保留下来。

1.2.1 核心痛点：为什么需要它？

在构建 RAG（检索增强生成）系统时，如果我们直接用 `RecursiveCharacterTextSplitter` 按固定长度（如 500 字符）切割一篇长篇 Markdown 技术文档，会导致一个致命问题：**上下文丢失**。

- 一个段落可能被从中间硬生生切断。
- 切割后的小块（Chunk）丢失了它所属的“章节”和“子章节”信息。当 LLM 检索到这个小块时，它根本不知道这段话是在讲“安装指南”还是“故障排除”。

`MarkdownHeaderTextSplitter` 的**解决方案**：它根据 Markdown 的标题（`#`，`##`，`###`）将文档切成逻辑完整的“区块（Sections）”，并将这些标题提取出来，存入 `metadata` 字典中。

1.2.2 基础用法与核心机制

1. 定义切割规则（`headers_to_split_on`）

你需要告诉分割器，你想根据哪些级别的标题进行切割，并为这些级别命名（作为 metadata 的 key）。

```
1 from langchain_text_splitters import MarkdownHeaderTextSplitter
2
3 markdown_document = "# Foo\n\n## Bar\n\nHi this is Jim\n\nHi this is Joe\n\n###\n\nBoo \n\nHi is Lance \n\n## Baz\n\nHi is Molly"
4
5 # 定义要监听的标题级别及其对应的元数据键名
6 headers_to_split_on = [
7     ("#", "Header 1"),
8     ("##", "Header 2"),
9     ("###", "Header 3"),
10 ]
11
12 markdown_splitter = MarkdownHeaderTextSplitter(headers_to_split_on)
13 md_header_splits = markdown_splitter.split_text(markdown_document)
```

2. 输出结果解析（层级继承机制）

分割器返回的是 LangChain 标准的 `Document` 对象列表。注意观察 `metadata` 的**层级继承**特性：

```
1 [
2     Document(metadata={'Header 1': 'Foo', 'Header 2': 'Bar'}, page_content='Hi is\nJim\nHi is Joe'),
3     Document(metadata={'Header 1': 'Foo', 'Header 2': 'Bar', 'Header 3': 'Boo'},\npage_content='Hi is Lance'),
4     Document(metadata={'Header 1': 'Foo', 'Header 2': 'Baz'}, page_content='Hi is\nMolly')
5 ]
```

- **亮点**：当切分到 `### Boo` 时，它不仅记录了当前的 `Header 3: 'Boo'`，还**自动继承**了父级的 `Header 1: 'Foo'` 和 `Header 2: 'Bar'`。这对于向量数据库检索时过滤上下文极其有用！

1.2.3 两大控制开关（进阶参数）

文档中介绍了两个非常实用的参数，用于微调分割行为：

1. `strip_headers` (是否剥离标题)
 - **默认行为 (True)**：分割器会把作为分割标记的标题从 `page_content`（正文内容）中删除，只保留在 `metadata` 中。这有助于减少传给 LLM 的冗余 Token。
 - **保留标题 (False)**：设置 `strip_headers=False` 后，标题文本会保留在 `page_content` 的开头。如果你希望 LLM 在阅读文本块时直接看到标题，可以使用此选项。
2. `return_each_line` (是否按行返回)
 - **默认行为 (False)**：分割器会将同一个标题下的**所有文本行聚合**成一个大的 `Document`。
 - **按行返回 (True)**：设置 `return_each_line=True` 后，该标题下的**每一行**都会变成一个独立的 `Document`，但每一行都会**共享**该标题的 `metadata`。这在某些需要极细粒度检索的场景下很有用。

1.2.4 最佳实践：黄金组合拳（解决长文本问题）

痛点： `MarkdownHeaderTextSplitter` 只按标题切分。如果某个章节（比如“# 第一章 绪论”）下面有 5000 字的纯文本，切出来的 `Document` 依然会超出 LLM 的上下文窗口限制。

解决方案： 将 `MarkdownHeaderTextSplitter` 与 `RecursiveCharacterTextSplitter` 串联使用。

1. **第一步（宏观）**：用 Markdown 分割器按标题切出“逻辑大块（Sections）”，并打上 Metadata 标签。
2. **第二步（微观）**：用递归字符分割器对这些“大块”进行二次切割，控制最终 Chunk 的长度。

```
1 from langchain_text_splitters import RecursiveCharacterTextSplitter
2
3 # 1. 宏观切分（带 Metadata）
4 markdown_splitter = MarkdownHeaderTextSplitter(headers_to_split_on,
5 strip_headers=False)
6 md_header_splits = markdown_splitter.split_text(markdown_document)
7
8 # 2. 微观切分（控制长度）
9 chunk_size = 250
10 chunk_overlap = 30
11 text_splitter = RecursiveCharacterTextSplitter(
12     chunk_size=chunk_size, chunk_overlap=chunk_overlap
13 )
14 # 关键注意：这里必须使用 split_documents 而不是 split_text!
15 # 这样才能保留第一步生成的 metadata!
16 splits = text_splitter.split_documents(md_header_splits)
```

1.2.5 避坑指南：关于 `chunk_overlap` 的真相

一个新手常问的疑惑：“为什么我设置了 `chunk_overlap`，但在跨章节的地方感觉没有重叠？”

原理解析：

- **Overlap（重叠）的作用域**：重叠机制只在单个 Section（章节）内部生效。当一个章节的内容太长，被 `RecursiveCharacterTextSplitter` 切成多个 Chunk 时，这些 Chunk 之间会有重叠。
- **绝不跨界**：Overlap 绝对不会跨越章节边界（例如从 `## 历史` 跨越到 `## 发展`）。因为不同章节在语义上是独立的，强行把“历史”的结尾和“发展”的开头重叠在一起，反而会污染上下文，降低检索质量。

- **API 注意：**在进行第二步组合切割时，务必调用 `text_splitter.split_documents(docs)`，而不是 `split_text`。只有 `split_documents` 才会把原文档的 `metadata`（标题信息）继承给二次切割后的小 Chunk。

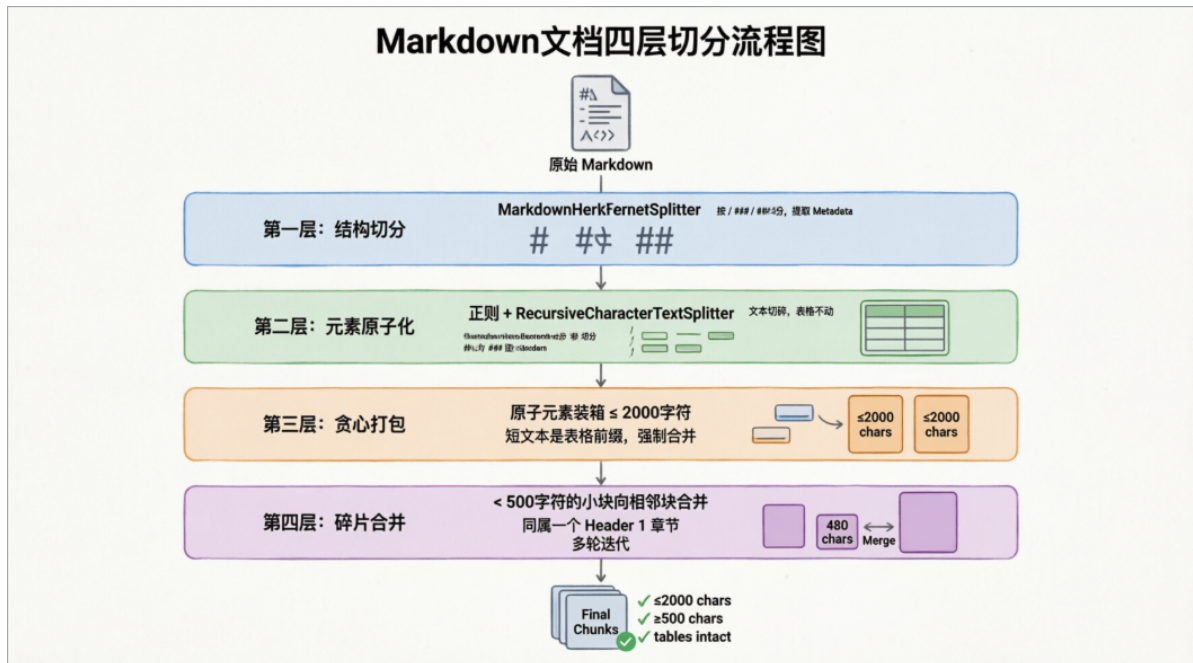
1.3 完整实现 - Vibe coding上手了

1.3.1 核心设计原则

一句话定义问题：将 Markdown 文档切分为适合 RAG 检索的文本块，同时满足三个约束：**块不能太大** (≤ 2000)、**块不能太小** (≥ 500)、**表格不能被切断**。

原则	含义
分层递进	每层只解决一个问题，不耦合
表格是原子	一旦识别为表格，后续所有操作都不触碰其内部
优先级明确	冲突时：表格完整 > 语义归属 > 长度上限 > 长度下限

1.3.2 四层管线



每层的核心理辑（一句话）

层	核心逻辑
第一层	遇到 # 就切一刀，标题路径存入 Metadata
第二层	用正则把 <table>...</table> 整体"抠出来"，剩下的文本用 RecursiveCharacterTextSplitter 切碎
第三层	逐个元素往"箱子"里塞，塞不下就封箱开新箱；如果箱里只剩短文本且新元素是表格，强制塞进去
第四层	扫一遍，发现小块就往前后看，同章节就合并；反复扫直到稳定

冲突场景	决策	原因
表格 3000 字符 > 上限 2000	豁免，保留完整表格	表格完整性 > 长度限制
短文本 20 字符 + 表格 2500 字符	合并为 2520 字符	短文本是表格的语义前缀
两个相邻小块属于不同 Header 2 但同 Header 1	允许合并	同属一个大章节，语义相关
两个小块属于不同 Header 1	不合并	跨章节合并会破坏语义

1.3.3 完整代码

```
1  import re
2  from langchain_core.documents import Document
3  from langchain_text_splitters import MarkdownHeaderTextSplitter,
   RecursiveCharacterTextSplitter
4
5  # ===== 全局配置 =====
6  MAX_LEN = 2000      # 块最大长度
7  MIN_LEN = 500       # 块最小长度
8  OVERLAP = 200       # 文本切分重叠
9  TABLE_RE = re.compile(r'<table.*?>/table>', re.IGNORECASE | re.DOTALL)
10
11
12 # ===== 第一层：按标题结构切分 =====
13 def split_by_headers(path):
14     """按 Markdown 标题切分，提取层级 Metadata"""
15     splitter = MarkdownHeaderTextSplitter(headers_to_split_on=[
16         ("#", "Header 1"), ("##", "Header 2"), ("###", "Header 3"),
17     ])
18     with open(path, "r", encoding="utf-8") as f:
19         return splitter.split_text(f.read())
20
21
22 # ===== 第二层：原子化 =====
23 def atomize(content, text_splitter):
24     """
25     将内容拆为原子序列：[文本块，表格，文本块，表格，...]
26     - 文本：用 RecursiveCharacterTextSplitter 切碎
27     - 表格：保持完整，作为一个原子
28     """
29     elements = []
30     last = 0
31     for m in TABLE_RE.finditer(content):
32         # 表格前的文本 -> 切碎
33         if m.start() > last:
34             text = content[last:m.start()]
35             if text.strip():
36                 elements.extend(text_splitter.split_text(text))
37         # 表格本身 -> 完整保留
38         elements.append(m.group(0))
39         last = m.end()
```

```

40     # 最后一个表格后的文本 → 切碎
41     if last < len(content) and content[last:].strip():
42         elements.extend(text_splitter.split_text(content[last:]))
43     return elements
44
45
46     # ===== 第三层：贪心打包 =====
47     # 贪心打包 = 顺序装箱：装得下就装，装不下就封箱换新箱；表格太大就单独一箱；箱里只剩"表名"小尾巴
48     # 时，宁可略微超载也要和表格同箱。
49     def pack(elements, metadata):
50         """
51         将原子元素贪心装箱：
52         - 正常情况：累积到 ≤ MAX_LEN
53         - 巨型表格：自身超限，单独成块
54         - 短文本 + 表格：强制合并（短文本是表格的语义前缀）
55         """
56         chunks, buf = [], ""
57
58         for elem in elements:
59             is_table = elem.strip().lower().startswith("<table")
60
61             # 巨型表格特例：自身就超限
62             if is_table and len(elem) > MAX_LEN:
63                 if buf and len(buf) < MIN_LEN:
64                     chunks.append(Document(page_content=buf + "\n\n" + elem,
65                                         metadata=metadata))
66                 else:
67                     if buf:
68                         chunks.append(Document(page_content=buf, metadata=metadata))
69                     chunks.append(Document(page_content=elem, metadata=metadata))
70                     buf = ""
71                     continue
72
73             # 常规装箱
74             sep = "\n\n" if buf else ""
75             candidate = buf + sep + elem
76
77             if len(candidate) <= MAX_LEN:
78                 buf = candidate # 没超限，继续累积
79             elif buf and len(buf) < MIN_LEN and is_table:
80                 buf = candidate # 短文本是表格前缀，强制合并
81             else:
82                 if buf:
83                     chunks.append(Document(page_content=buf, metadata=metadata))
84                 buf = elem # 开新箱
85
86             if buf:
87                 chunks.append(Document(page_content=buf, metadata=metadata))
88         return chunks
89
90     # ===== 第四层：碎片合并 =====
91     def same_section(a, b):
92         """判断两个块是否可合并（同 Metadata 或同 Header 1）"""
93         ma, mb = a.metadata or {}, b.metadata or {}
94         if ma == mb:
95             return True
96         h1a, h1b = ma.get('Header 1', ''), mb.get('Header 1', '')

```

```

96     if h1a and h1a == h1b:
97         return True
98     return not ma or not mb
99
100
101 def merge_ok(text):
102     """合并后是否合法: 不超长, 或含表格豁免"""
103     return len(text) <= MAX_LEN or "<table" in text.lower()
104
105
106 def pick_meta(a, b):
107     """合并后取更具体的 Metadata (key 更多的)"""
108     if not a: return b
109     if not b: return a
110     return a if len(a) >= len(b) else b
111
112
113 def merge_small(chunks):
114     """多轮迭代: 将 < MIN_LEN 的小块与相邻同章节块合并"""
115     for _ in range(10):
116         result, changed = [], False
117         i = 0
118         while i < len(chunks):
119             c = chunks[i]
120
121             # 已经够大, 直接保留
122             if len(c.page_content) >= MIN_LEN:
123                 result.append(c)
124                 i += 1
125                 continue
126
127             # 尝试向后合并
128             if i + 1 < len(chunks) and same_section(c, chunks[i + 1]):
129                 txt = c.page_content + "\n\n" + chunks[i + 1].page_content
130                 if merge_ok(txt):
131                     meta = pick_meta(c.metadata, chunks[i + 1].metadata)
132                     result.append(Document(page_content=txt, metadata=meta))
133                     i += 2
134                     changed = True
135                     continue
136
137             # 尝试向前合并
138             if result and same_section(result[-1], c):
139                 txt = result[-1].page_content + "\n\n" + c.page_content
140                 if merge_ok(txt):
141                     meta = pick_meta(result[-1].metadata, c.metadata)
142                     result[-1] = Document(page_content=txt, metadata=meta)
143                     i += 1
144                     changed = True
145                     continue
146
147             # 无法合并, 保留
148             result.append(c)
149             i += 1
150
151     chunks = result
152     if not changed:
153         break

```

```

154         return chunks
155
156
157     # ===== 主流程（一行调用） =====
158     def smart_split(path):
159         """完整的四层切分管线"""
160         # 第一层
161         docs = split_by_headers(path)
162         # 第二、三层
163         text_splitter = RecursiveCharacterTextSplitter(chunk_size=MAX_LEN,
164 chunk_overlap=OVERLAP)
165         chunks = []
166         for doc in docs:
167             elements = atomize(doc.page_content, text_splitter)
168             chunks.extend(pack(elements, doc.metadata))
169         # 第四层
170         return merge_small(chunks)
171
172     # ===== 执行与验证 =====
173     if __name__ == "__main__":
174         final_chunks = smart_split(r"result\sample\auto\sample_new.md")
175
176         print(f"✅ 切分完成! 共 {len(final_chunks)} 个 Chunk\n")
177         for i, c in enumerate(final_chunks):
178             n = len(c.page_content)
179             has_tbl = "<table" in c.page_content.lower()
180             status = "✅" if MIN_LEN <= n <= MAX_LEN else ("⚠️" if has_tbl and n >
MAX_LEN else "❌")
181             print(f"Chunk {i+1:3d} | {status} | {n:5d}字符 | 表格:{'是' if has_tbl
else '否'} | {c.metadata}")

```

记住一句话：先保结构（Metadata），再保表格（原子化），再控大小（装箱 + 合并）。

1.4 企业级切分实现（手动切块，仅供参考）

1.4.1 目标

- 按 Markdown 标题结构将文档切分为语义完整的文本块
- 为每个 chunk 追踪其父级标题（parent_title），便于同宗同源合并
- 控制每个文本块的长度在合理范围内（不超过 max_length）
- 避免产生过短的碎片化文本块（小于 min_length 时合并）
- 保留标题与正文的结构关系，便于后续检索和展示

1.4.2 需求分析

输入：

- `md_content`：Markdown 文档内容
- `file_title`：文件标题（用于标记来源）

输出：

- `chunks`：切分后的文本块列表，每个 chunk 包含：
 - `title`：章节标题
 - `content`：完整内容（标题 + 正文）
 - `file_title`：来源文件标题

- `parent_title`：父标题（层级追踪产生，用于合并判断）
- `part`（可选）：片段编号（二次切分时产生）

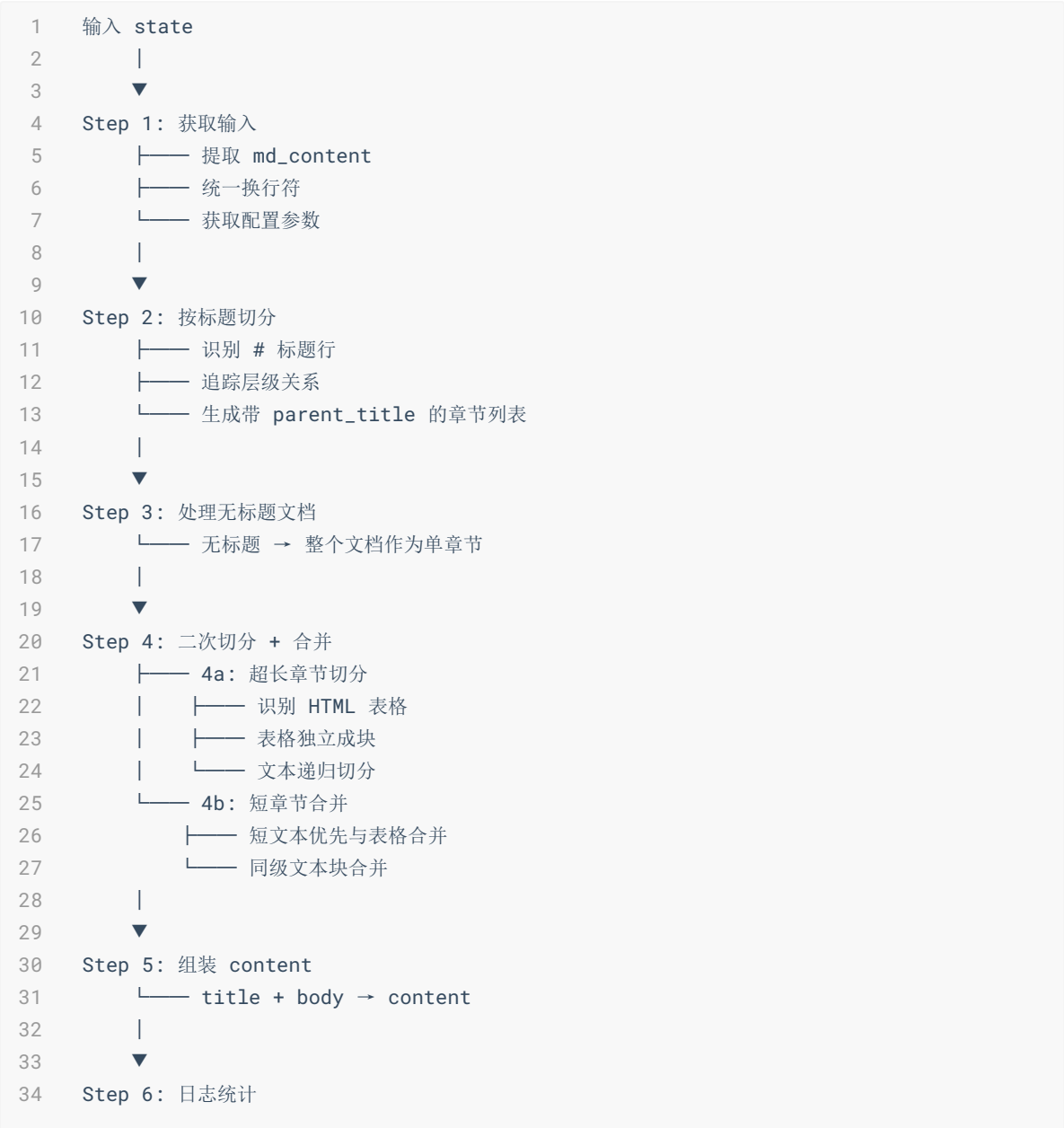
配置参数：

- `max_content_length`：单个 chunk 的最大字符数（默认 500）
- `min_content_length`：触发合并的最小字符数（默认 100）

边界条件：

场景	处理方式
<code>md_content</code> 为空	抛出 <code>DocumentSplitError</code>
全文无标题	整体作为一个 chunk，标题设为 "无标题"， <code>parent_title</code> 为 <code>file_title</code>
章节超长	使用 LangChain RecursiveCharacterTextSplitter 二次切分
章节过短	与下一个同 <code>parent_title</code> 的章节合并
代码块内有 <code>#</code>	不识别为标题（代码围栏检测）

1.4.3 完整处理流程



```
35     └── 输出切分统计信息
36     |
37     ▼
38 Step 7: 备份
39     └── 保存为 chunks.json
40     |
41     ▼
42 输出 state (含 chunks)
```

1.4.4 具体实现步骤

Step 1: 获取输入

功能描述:

从状态字典中获取 Markdown 内容和配置参数，统一换行符格式。

实现要点:

1. 获取 md_content

- 从 `state.get("md_content", "")` 获取内容
- 如果内容为空，后续步骤会自然产生空结果

2. 统一换行符

- Windows 换行符 `\r\n` 转为 `\n`
- Mac 旧版换行符 `\r` 转为 `\n`
- 确保后续按行处理时行为一致

3. 获取配置

- `file_title`：来源文件标题，用于无标题段落的标识
- `max_length`：从全局配置获取最大切片长度（默认 2000）

代码片段:

```
1 def _get_inputs(state):
2     logger.info("step_1:获取输入")
3     content = state.get("md_content", "")
4     if content:
5         content = content.replace("\r\n", "\n").replace("\r", "\n")
6
7     file_title = state.get("file_title", "")
8     return content, file_title, max_content_length
```

输入示例:

```
1 state = {
2     "md_content": "# 标题\r\n正文内容\r\n## 子标题\r\n更多内容",
3     "file_title": "sample.md"
4 }
```

输出示例:

```
1 content = "# 标题\n正文内容\n## 子标题\n更多内容"
2 file_title = "sample.md"
3 max_length = 2000
```

Step 2: 按标题一级切分（带层级追踪）

功能描述:

按 Markdown 标题行（`# ~ #####`）将文档切分为章节，同时追踪每个章节的父标题层级关系。

实现要点:

1. 标题识别

- 使用正则 `^\s*({1,6})\s+(.+)` 匹配标题行
- 提取标题级别（1-6）和标题文本
- 代码围栏（``` 或 `~~~`）内的 `#`` 不会被误识别

2. 层级追踪

- 使用 `hierarchy` 数组（索引 1-6）记录各级标题的最新足迹
- 遇到新标题时，清除该级别以下的所有子标题记录
- 每个章节通过向上查找 `hierarchy` 获取 `parent_title`

3. 内容收集

- 标题行触发 `_flush()` 保存上一个章节
- 非标题行累积到 `body_lines` 列表
- 文档末尾自动触发最后一次保存

4. parent_title 计算

- 从当前标题级别-1 开始向上查找
- 找到最近的非空父标题作为 `parent_title`
- 如果没有父标题，使用当前标题或文件标题

代码片段:

```
1 def _split_by_headings(content: str, file_title: str) -> Tuple[List[dict], bool]:
2     logger.info("step_2: 按标题切分并追踪层级")
3
4     heading_re = re.compile(r"^\s*({1,6})\s+(.+)")
5     lines = content.split("\n")
6
7     sections: List[dict] = []
8     current_title = ""
9     current_level = 0
10    body_lines: List[str] = []
11    has_title = False
12    in_fence = False
13
14    hierarchy = [""] * 7
15
16    def _flush():
17        body = "\n".join(body_lines).strip()
18        if current_title or body:
19            parent_title = next(
20                (hierarchy[lvl] for lvl in range(current_level - 1, 0, -1) if
21                hierarchy[lvl]),
22                current_title or file_title
23            )
24            sections.append({
25                "title": current_title,
26                "body": body,
27                "file_title": file_title,
28                "parent_title": parent_title,
```

```

29         "level": current_level,
30     })
31
32     for line in lines:
33         if line.strip().startswith("` ` `") or line.strip().startswith("~~~"):
34             in_fence = not in_fence
35
36         match = heading_re.match(line) if not in_fence else None
37
38         if match:
39             has_title = True
40             _flush()
41
42             level = len(match.group(1))
43             current_level = level
44             current_title = line.strip()
45             hierarchy[level] = current_title
46
47             for i in range(level + 1, 7):
48                 hierarchy[i] = ""
49
50             body_lines = []
51         else:
52             body_lines.append(line)
53
54     _flush()
55     return sections, has_title

```

输入示例:

```

1  # 第一章
2  这是第一章的内容。
3
4  ## 第一节
5  这是第一节的内容。
6
7  # 第二章
8  这是第二章的内容。

```

输出示例:

```

1  sections = [
2      {
3          "title": "# 第一章",
4          "body": "这是第一章的内容。",
5          "parent_title": "# 第一章",
6          "level": 1
7      },
8      {
9          "title": "## 第一节",
10         "body": "这是第一节的内容。",
11         "parent_title": "# 第一章", # 向上追溯到 H1
12         "level": 2
13     },
14     {
15         "title": "# 第二章",
16         "body": "这是第二章的内容。",
17         "parent_title": "# 第二章",

```

```
18         "level": 1
19     }
20 ]
21 has_title = True
```

Step 3: 处理无标题情况

功能描述:

如果文档没有任何标题，将整个文档作为一个单独的章节处理。

实现要点:

- 检查 Step 2 返回的 `has_title` 标记
- 无标题时将整个内容放入一个章节
- `parent_title` 使用文件标题

```
1 if not has_title:
2     sections = [{
3         "title": "无标题",
4         "body": content,
5         "file_title": file_title,
6         "parent_title": file_title
7     }]
8     logger.info("全文无标题，作为单个 chunk 处理")
```

触发条件:

- 文档中不存在任何 `#` 开头的标题行
- 常见于纯文本文档或格式不规范的文档

Step 4: 二次切分 + 合并短章节

功能描述:

对超长章节进行二次切分（保护表格完整性），然后合并过短的相邻章节。

实现要点:

1. 超长章节切分

- 判断章节长度是否超过 `max_length`
- 使用 `<table>...</table>` 正则将内容分割为"文本片段"和"表格片段"
- 表格片段直接独立成块（标记 `has_table: True`）
- 文本片段使用 `RecursiveCharacterTextSplitter` 进一步切分

2. 短章节合并

- 第一步：表格前面的短文本优先与表格合并
- 第二步：常规文本块之间的合并
- 表格块不参与常规合并
- 合并时继承更有意义的标题

代码片段:

4a: 超长章节切分

```
1 def _split_long_section(section: dict, max_length: int) -> List[dict]:
2     title = section.get("title", "")
3     body = section.get("body", "")
4     file_title = section.get("file_title", "")
5     parent_title = section.get("parent_title", title)
```

```

6
7     if len(body) + len(title) <= max_length:
8         return [section]
9
10    pattern = re.compile(r'(<table>.*?</table>)', re.DOTALL | re.IGNORECASE)
11    parts = pattern.split(body)
12
13    if len(parts) <= 1:
14        return _split_text_block(body, title, file_title, parent_title,
max_length)
15
16    chunks = []
17    part_num = 0
18
19    for part in parts:
20        if not part.strip():
21            continue
22
23        part_num += 1
24
25        if _is_html_table_block(part):
26            chunks.append({
27                "title": f"{title}-table" if title else "table",
28                "body": part.strip(),
29                "file_title": file_title,
30                "parent_title": parent_title,
31                "part": part_num,
32                "has_table": True,
33            })
34        else:
35            text_chunks = _split_text_block(
36                part.strip(), title, file_title, parent_title,
37                max_length, part_num - 1
38            )
39            chunks.extend(text_chunks)
40            part_num += len(text_chunks) - 1
41
42    for i, chunk in enumerate(chunks):
43        chunk["part"] = i + 1
44
45    return chunks if chunks else [section]

```

4b: 短章节合并

```

1    def _merge_short_sections(sections: List[dict], min_length: int) -> List[dict]:
2        if not sections:
3            return []
4
5        # 第一步: 表格前面的短文本与表格合并
6        merged_with_tables = []
7        i = 0
8        while i < len(sections):
9            current = sections[i]
10
11            if (i + 1 < len(sections)
12                and sections[i + 1].get("has_table")
13                and not current.get("has_table")
14                and len(current.get("body", "")) < min_length):

```

```

15
16         table_sec = sections[i + 1]
17         table_sec["body"] = f"
{current['body']}\n\n{table_sec['body']}".strip()
18
19         if current.get("title") and not current["title"].endswith("-table"):
20             table_sec["title"] = current["title"]
21
22         merged_with_tables.append(table_sec)
23         i += 2
24         continue
25
26     merged_with_tables.append(current)
27     i += 1
28
29     # 第二步：常规文本块合并
30     merged = []
31     current = merged_with_tables[0]
32
33     for next_sec in merged_with_tables[1:]:
34         if _should_merge(current, next_sec, min_length):
35             current["body"] = f"{current['body']}\n\n{next_sec['body']}".strip()
36             if next_sec.get("title") and "#" not in next_sec["title"]:
37                 current["title"] = next_sec["title"]
38             else:
39                 merged.append(current)
40                 current = next_sec
41
42     merged.append(current)
43     return merged

```

输入示例（超长章节含表格）：

```

1  ## 收支总表
2
3  这是收支总表的说明文字，比较长...
4
5  部门公开表 1
6  单位：万元
7  <table>...超长表格...</table>

```

输出示例：

```

1  # 说明文字+表格被合并为一个 chunk
2  {
3      "title": "## 收支总表",
4      "body": "这是收支总表的说明文字...\n\n部门公开表 1\n单位：万元\n\n<table>...
</table>",
5      "has_table": False, # 混合内容不标记
6  }

```

Step 5: 组装最终 content

功能描述:

将内部使用的 `title` 和 `body` 字段组装为最终的 `content` 字段, 保留溯源信息。

实现要点:

1. `content = title + "\n\n" + body` (当两者都存在时)
2. 仅保留必要的外部字段
3. 保留 `parent_title`、`part`、`has_table` 供下游使用

代码片段:

```
1  def _assemble_content(sections: List[dict]) -> List[dict]:
2      logger.info("step_5: 组装 content")
3
4      result: List[dict] = []
5      for sec in sections:
6          title = sec.get("title", "")
7          body = sec.get("body", "")
8
9          content = f"{title}\n\n{body}".strip() if title and body else (title or
body)
10
11         chunk = {
12             "title": title,
13             "content": content,
14             "file_title": sec.get("file_title", ""),
15         }
16
17         if "parent_title" in sec:
18             chunk["parent_title"] = sec["parent_title"]
19         if "part" in sec:
20             chunk["part"] = sec["part"]
21         if "has_table" in sec:
22             chunk["has_table"] = sec["has_table"]
23
24         result.append(chunk)
25
26     return result
```

输入示例:

```
1  {
2      "title": "## 收支总表",
3      "body": "说明文字\n\n<table>...</table>",
4      "file_title": "sample.md",
5      "parent_title": "# 第一章",
6      "part": 1,
7      "has_table": True
8  }
```

输出示例:

```

1  {
2      "title": "## 收支总表",
3      "content": "## 收支总表\n\n说明文字\n\n<table>...</table>",
4      "file_title": "sample.md",
5      "parent_title": "# 第一章",
6      "part": 1,
7      "has_table": True
8  }

```

Step 6: 日志统计

功能描述:

输出切分统计信息，便于调试和监控。

输出内容:

- 原文档行数
- 最终切分章节数
- 最大切片长度
- 前 5 个章节标题预览

```

1  def _log_summary(self, raw_content, sections, max_length):
2      self.log_step("step_6", "输出统计")
3
4      lines_count = raw_content.count("\n") + 1
5      self.logger.info(f"原文档行数: {lines_count}")
6      self.logger.info(f"最终切分章节数: {len(sections)}")
7      self.logger.info(f"最大切片长度: {max_length}")
8
9      if sections:
10         self.logger.info("章节预览:")
11         for i, sec in enumerate(sections[:5]):
12             title = sec.get("title", "")[:50]
13             self.logger.info(f"  {i + 1}. {title}...")
14         if len(sections) > 5:
15             self.logger.info(f" ... 还有 {len(sections) - 5} 个章节")

```

Step 7: 备份切片

功能描述:

将切分结果保存为 JSON 文件，便于调试和下游使用。

实现要点:

- 从 state 获取输出目录
- 创建目录（如不存在）
- 以 UTF-8 编码写入 JSON

```

1  def _backup_chunks(self, state, sections):
2      self.log_step("step_7", "备份切片")
3
4      # 优先使用 file_dir, 兼容 local_dir
5      local_dir = state.get("file_dir", state.get("local_dir", ""))
6      if not local_dir:
7          self.logger.debug("未设置 file_dir/local_dir, 跳过备份")
8          return
9

```

```

10     try:
11         os.makedirs(local_dir, exist_ok=True)
12         output_path = os.path.join(local_dir, "chunks.json")
13         with open(output_path, "w", encoding="utf-8") as f:
14             json.dump(sections, f, ensure_ascii=False, indent=2)
15         self.logger.info(f"已备份到: {output_path}")
16     except Exception as e:
17         self.logger.warning(f"备份失败: {e}")

```

1.4.5 具体实现

```

1  import logging
2  import os
3  from typing import List, Tuple
4  import re
5  from langchain_text_splitters import RecursiveCharacterTextSplitter
6  import json
7
8  logging.basicConfig(
9      level=logging.DEBUG,
10     format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
11 )
12 logger = logging.getLogger("mineru")
13
14 # ===== 文档处理配置 =====
15 max_content_length: int = 2000 # 切片最大长度
16 min_content_length: int = 500 # 合并短内容的最小长度
17 overlap_sentences: int = 1 # 句子级切分时的重叠句数
18 #item_name_chunk_k: int = 3 # 商品名识别时使用的切片数量
19
20
21 def _is_html_table_block(text: str) -> bool:
22     """
23     判断文本块是否包含 HTML 表格
24     """
25     return '<table>' in text and '</table>' in text
26
27
28 # ----- #
29 #                               Step 1: 获取输入                               #
30 # ----- #
31 def _get_inputs(state):
32     logger.info("step_1:获取输入")
33
34     content = state.get("md_content", "")
35     if content:
36         content = content.replace("\r\n", "\n").replace("\r", "\n")
37
38     file_title = state.get("file_title", "")
39
40     return content, file_title, max_content_length
41
42
43 # ----- #
44 #                               Step 2: 按标题一级切分 (带层级追踪)                               #
45 # ----- #
46 def _split_by_headings(content: str, file_title: str) -> Tuple[List[dict],
bool]:

```

```

47     """
48     按 Markdown 标题行切分，向上追踪层级找 parent_title
49     """
50     logger.info("step_2: 按标题切分并追踪层级")
51
52     heading_re = re.compile(r"^\s*({1,6})\s+(.+)")
53     lines = content.split("\n")
54
55     sections: List[dict] = []
56     current_title = ""
57     current_level = 0
58     body_lines: List[str] = []
59     has_title = False
60     in_fence = False
61
62     # 记录 1-6 级标题的最新足迹
63     hierarchy = [""] * 7
64
65     def _flush():
66         body = "\n".join(body_lines).strip()
67         if current_title or body:
68             # 向上找最近的非空父标题
69             parent_title = next(
70                 (hierarchy[lvl] for lvl in range(current_level - 1, 0, -1) if
hierarchy[lvl]),
71                 current_title or file_title
72             )
73
74             sections.append({
75                 "title": current_title,
76                 "body": body,
77                 "file_title": file_title,
78                 "parent_title": parent_title,
79                 "level": current_level,
80             })
81
82     for line in lines:
83         # 代码围栏保护
84         if line.strip().startswith("```") or line.strip().startswith("~~~"):
85             in_fence = not in_fence
86
87         match = heading_re.match(line) if not in_fence else None
88
89         if match:
90             has_title = True
91             _flush()
92
93             level = len(match.group(1))
94             current_level = level
95             current_title = line.strip()
96             hierarchy[level] = current_title
97
98             # 清除当前标题下的所有子标题足迹
99             for i in range(level + 1, 7):
100                 hierarchy[i] = ""
101
102             body_lines = []
103         else:

```

```

104         body_lines.append(line)
105
106     _flush()
107     return sections, has_title
108
109
110 # ----- #
111 #           Step 4: 二次切分 + 合并短章节           #
112 # ----- #
113 def _split_and_merge(
114     sections: List[dict],
115     max_length: int,
116     min_length: int,
117 ) -> List[dict]:
118     logger.info("step_4: 二次切分和合并")
119
120     if max_length <= 0:
121         return sections
122
123     # 切分超长章节
124     split_result: List[dict] = []
125     for section in sections:
126         split_result.extend(_split_long_section(section, max_length))
127
128     # 合并过短章节
129     return _merge_short_sections(split_result, min_length)
130
131
132 def _split_long_section(section: dict, max_length: int) -> List[dict]:
133     """
134     切分超长章节, 遇到 HTML 表格直接独立成块
135     """
136     title = section.get("title", "")
137     body = section.get("body", "")
138     file_title = section.get("file_title", "")
139     parent_title = section.get("parent_title", title)
140
141     # 不超长, 直接返回
142     if len(body) + len(title) <= max_length:
143         return [section]
144
145     # 用表格边界切分
146     pattern = re.compile(r'(<table>.*?</table>)', re.DOTALL | re.IGNORECASE)
147     parts = pattern.split(body)
148
149     if len(parts) <= 1:
150         # 无表格, 纯文本切分
151         return _split_text_block(body, title, file_title, parent_title,
max_length)
152
153     # 处理每个片段
154     chunks = []
155     part_num = 0
156
157     for part in parts:
158         if not part.strip():
159             continue
160

```

```

161         part_num += 1
162
163     if _is_html_table_block(part):
164         # 表格独立成块
165         chunks.append({
166             "title": f"{title}-table" if title else "table",
167             "body": part.strip(),
168             "file_title": file_title,
169             "parent_title": parent_title,
170             "part": part_num,
171             "has_table": True,
172         })
173     else:
174         # 文本片段，可能还需要切分
175         text_chunks = _split_text_block(
176             part.strip(), title, file_title, parent_title,
177             max_length, part_num - 1
178         )
179         chunks.extend(text_chunks)
180         part_num += len(text_chunks) - 1
181
182     # 重新编号
183     for i, chunk in enumerate(chunks):
184         chunk["part"] = i + 1
185
186     return chunks if chunks else [section]
187
188
189 def _split_text_block(
190     text: str,
191     title: str,
192     file_title: str,
193     parent_title: str,
194     max_length: int,
195     start_part: int = 0
196 ) -> List[dict]:
197     """
198     切分纯文本块（不含表格）
199     """
200     title_prefix = f"{title}\n\n" if title else ""
201     available = max_length - len(title_prefix)
202
203     # 不超长或无法切分，直接返回
204     if len(text) <= available or available <= 0:
205         return [{
206             "title": title if title else "chunk-1",
207             "body": text,
208             "file_title": file_title,
209             "parent_title": parent_title,
210             "part": start_part + 1,
211             "has_table": False,
212         }]
213
214     # 使用标准切分器
215     splitter = RecursiveCharacterTextSplitter(
216         chunk_size=available,
217         chunk_overlap=0,

```

```

218         separators=["\n\n", "\n", "。", "!", "?", ";", ".", "!", "?", ";", "
"]
219     )
220
221     pieces = splitter.split_text(text)
222
223     chunks = []
224     for i, piece in enumerate(pieces):
225         chunks.append({
226             "title": f"{title}-{i + 1}" if title else f"chunk-{start_part + i +
1}",
227             "body": piece.strip(),
228             "file_title": file_title,
229             "parent_title": parent_title,
230             "part": start_part + i + 1,
231             "has_table": False,
232         })
233
234     return chunks
235
236
237 def _should_merge(current: dict, next_sec: dict, min_length: int) -> bool:
238     """判断是否应该合并两个块"""
239     # 任一包含表格，不合并
240     if current.get("has_table") or next_sec.get("has_table"):
241         return False
242
243     # 不在同一父标题下，不合并
244     if current.get("parent_title") != next_sec.get("parent_title"):
245         return False
246
247     # 当前块长度足够，不合并
248     if len(current.get("body", "")) >= min_length:
249         return False
250
251     return True
252
253
254 def _merge_short_sections(sections: List[dict], min_length: int) -> List[dict]:
255     """
256     优化版合并逻辑：
257     1. 表格前面的短文本优先和表格合并
258     2. 表格块不参与常规合并
259     3. 合并时继承前面文本的标题
260     """
261     if not sections:
262         return []
263
264     # 第一步：将表格前面的短文本与表格合并
265     merged_with_tables = []
266     i = 0
267     while i < len(sections):
268         current = sections[i]
269
270         # 检查是否需要将当前文本合并到后面的表格
271         if (i + 1 < len(sections)
272             and sections[i + 1].get("has_table")
273             and not current.get("has_table"))

```

```

274         and len(current.get("body", "")) < min_length):
275
276         table_sec = sections[i + 1]
277         table_sec["body"] = f"
{current['body']}\n\n{table_sec['body']}".strip()
278
279         # 用前面文本的标题替换表格的默认标题（如果更有意义）
280         if current.get("title") and not current["title"].endswith("-table"):
281             table_sec["title"] = current["title"]
282
283         merged_with_tables.append(table_sec)
284         i += 2
285         continue
286
287         merged_with_tables.append(current)
288         i += 1
289
290     # 第二步：常规合并文本块
291     merged = []
292     current = merged_with_tables[0]
293
294     for next_sec in merged_with_tables[1:]:
295         if _should_merge(current, next_sec, min_length):
296             # 合并
297             current["body"] = f"{current['body']}\n\n{next_sec['body']}".strip()
298             # 保留更有意义的标题（非一级标题的次级标题）
299             if next_sec.get("title") and "#" not in next_sec["title"]:
300                 current["title"] = next_sec["title"]
301             else:
302                 merged.append(current)
303                 current = next_sec
304
305     merged.append(current)
306     return merged
307
308
309     # ----- #
310     #           Step 5: 组装最终 content           #
311     # ----- #
312     def _assemble_content(sections: List[dict]) -> List[dict]:
313         """
314         将 title + body 组装为最终的 content 字段
315         """
316         logger.info("step_5: 组装 content")
317
318         result: List[dict] = []
319         for sec in sections:
320             title = sec.get("title", "")
321             body = sec.get("body", "")
322
323             content = f"{title}\n\n{body}".strip() if title and body else (title or
body)
324
325             chunk = {
326                 "title": title,
327                 "content": content,
328                 "file_title": sec.get("file_title", ""),
329             }

```

```

330
331     # 保留溯源信息
332     if "parent_title" in sec:
333         chunk["parent_title"] = sec["parent_title"]
334     if "part" in sec:
335         chunk["part"] = sec["part"]
336     if "has_table" in sec:
337         chunk["has_table"] = sec["has_table"]
338
339     result.append(chunk)
340
341     return result
342
343
344     # ----- #
345     #                               日志 & 备份                               #
346     # ----- #
347     def _log_summary(raw_content: str, sections: List[dict], max_length: int):
348         logger.info("step_6: 日志与输出统计")
349
350         lines_count = raw_content.count("\n") + 1
351         logger.info(f"原文档行数: {lines_count}")
352         logger.info(f"最终切分章节数: {len(sections)}")
353         logger.info(f"最大切片长度: {max_length}")
354
355         if sections:
356             logger.info("章节预览:")
357             for i, sec in enumerate(sections[:5]):
358                 title = sec.get("title", "")[:50]
359                 logger.info(f"  {i + 1}. {title}...")
360             if len(sections) > 5:
361                 logger.info(f"    ... 还有 {len(sections) - 5} 个章节")
362
363
364     def _backup_chunks(state, sections: List[dict]):
365         logger.info("step_7: 备份切片")
366
367         local_dir = state.get("file_dir", state.get("local_dir", ""))
368         if not local_dir:
369             logger.debug("未设置 file_dir/local_dir, 跳过备份")
370             return
371
372         try:
373             os.makedirs(local_dir, exist_ok=True)
374             output_path = os.path.join(local_dir, "chunks.json")
375             with open(output_path, "w", encoding="utf-8") as f:
376                 json.dump(sections, f, ensure_ascii=False, indent=2)
377             logger.info(f"已备份到: {output_path}")
378         except Exception as e:
379             logger.warning(f"备份失败: {e}")
380
381
382     # ===== 主流程 =====
383
384     def process_markdown(state: dict) -> dict:
385         """
386         完整的 Markdown 文档处理流程
387

```

```

388     Args:
389         state: 包含 md_content, file_title 的字典
390
391     Returns:
392         更新后的 state, 包含 chunks 字段
393     """
394     # Step 1: 获取输入
395     content, file_title, max_length = _get_inputs(state)
396
397     # Step 2: 按标题切分
398     sections, has_title = _split_by_headings(content, file_title)
399
400     # Step 3: 处理全文无标题的情况
401     if not has_title:
402         sections = [{
403             "title": "无标题",
404             "body": content,
405             "file_title": file_title,
406             "parent_title": file_title
407         }]
408         logger.info("全文无标题, 作为单个 chunk 处理")
409
410     # Step 4: 二次切分 + 合并
411     sections = _split_and_merge(sections, max_length, min_content_length)
412
413     # Step 5: 组装最终 content
414     sections = _assemble_content(sections)
415
416     # Step 6: 日志统计
417     _log_summary(content, sections, max_length)
418
419     # Step 7: 备份
420     state["chunks"] = sections
421     _backup_chunks(state, sections)
422
423     return state
424
425
426 if __name__ == "__main__":
427     sample_document_path = r"result\sample\auto\sample_new.md"
428
429     with open(sample_document_path, 'r', encoding='utf-8') as f:
430         content = f.read().strip()
431
432     state = {
433         "file_title": "sample_new",
434         "md_content": content,
435         "file_dir": os.path.dirname(sample_document_path)
436     }
437
438     # 执行完整流程
439     state = process_markdown(state)
440
441     print("\n" + "=" * 50)
442     print("切片执行完毕, 最终状态字典概览: ")
443     print("=" * 50)
444
445     preview_chunks = state.get("chunks", [])[:10]

```

```
446     print(json.dumps(preview_chunks, ensure_ascii=False, indent=4))
447     print(f"\n..... (共生成 {len(state.get('chunks', []))} 个 Chunks, 详情请查看
      chunks.json)")
```

核心原则可以概括为：

标题负责语义边界，最大长度负责模型输入限制，最小长度负责避免内容过碎，父标题负责保留章节上下文。

1.5 Markdown 与 HTML 表格处理终极指南（补充）

在构建高质量的 RAG（检索增强生成）系统时，使用文本切分器直接处理包含表格的文档，是一场极其惨烈的灾难。切分器会无视表格的二维结构，将 `|` 或 `<tr><td>` 暴力斩断，导致下游的大模型（LLM）产生严重的幻觉和语义丢失。

针对这一“**世界级痛点**”，工业界常见有 4 个层级的演进式解决方案。

1.5.1 方案介绍

方案一：表格隔离保护法

核心思想：将表格视为“不可侵犯的原子实体”。

实现方式：

在文本进入通用切分器之前，使用正则表达式（如 `(\n?(?:\|[\^\\n]+\|)|\n)+`）将标准的 Markdown 表格整个提取出来，作为独立的 Chunk 存放，剩余的正文再送入切分器。

- **优点：**实现简单，能有效保护小型标准表格的完整性。
- **缺点：**如果表格本身超出了 Embedding 模型的 Token 上限，依然会面临超载报错；对非标准的 HTML 表格无能为力。

方案二：表头续传切分法

核心思想：表格可以切，但必须保证每一块“残骸”都拥有完整的身份证。

实现方式：

编写专门的表格切分算法。先提取表格的首行（表头），然后按固定行数切分下方的数据行。在组装 Chunk 时，强制将提取出的表头拼接 to 每一个数据子块的顶部。

- **优点：**完美解决了超大表格的容量超载问题，LLM 拿到子块依然能看懂列含义。
- **缺点：**产物依然是二维结构（带有 `|` 管道符）。大部分向量模型（如 BGE-M3）对空间符号的敏感度极差，导致检索命中率依然不理想。

方案三：降维转译法

核心思想：消灭二维表格，将表格拍平，转译为高信息密度的自然语言。

实现方式：

这是目前大厂处理复杂图文文档的核心策略。

1. **矩阵投影：**引入 `BeautifulSoup` 等解析器，彻底解析 HTML/Markdown 表格。针对极其恶心的 `rowspan` 和 `colspan`（跨行跨列），在内存中构建二维数组（2D List），并将合并的单元格内容进行**向下、向右的物理填充**。
2. **意图嗅探：**在次判断表格类型（左上角空置交叉表、无表头 K-V 表等）。
3. **语义重构：**将每一行数据拼接成标准中文，例如：
- **【直流电压】（对应功能）：**量程为20V，精确度为±(0.5%)。

- **优点：**
 - **绝对安全：**转译成普通句子后，随意被切分也不会丢失语义。

- **极度精准**：向量模型对自然语言的 Embedding 效果成倍提升，召回率暴增。
- **缺点**：算法实现极其复杂，极度考验开发者对各类“变态排版”的兼容能力。

方案四: VLM 视觉降维引擎

核心思想：放弃代码解析，把排版解析工作外包给拥有“视觉”的 AI 智能体。

实现方式：

针对**多级嵌套表头**、**表内嵌表**、**跨页断裂表**等无法用代码穷举的极端脏数据，引入“动态路由”机制：

1. **渲染截图**：使用 `html2image` 或 `Playwright` 将复杂的 HTML 表格渲染并在后台截图。
2. **多模态提取**：将 Base64 图片喂给 GPT-4o 或 Gemini-1.5-Pro 等视觉大模型。
3. **Prompt 约束**：要求 VLM 输出展平后的一维自然语言键值对描述。

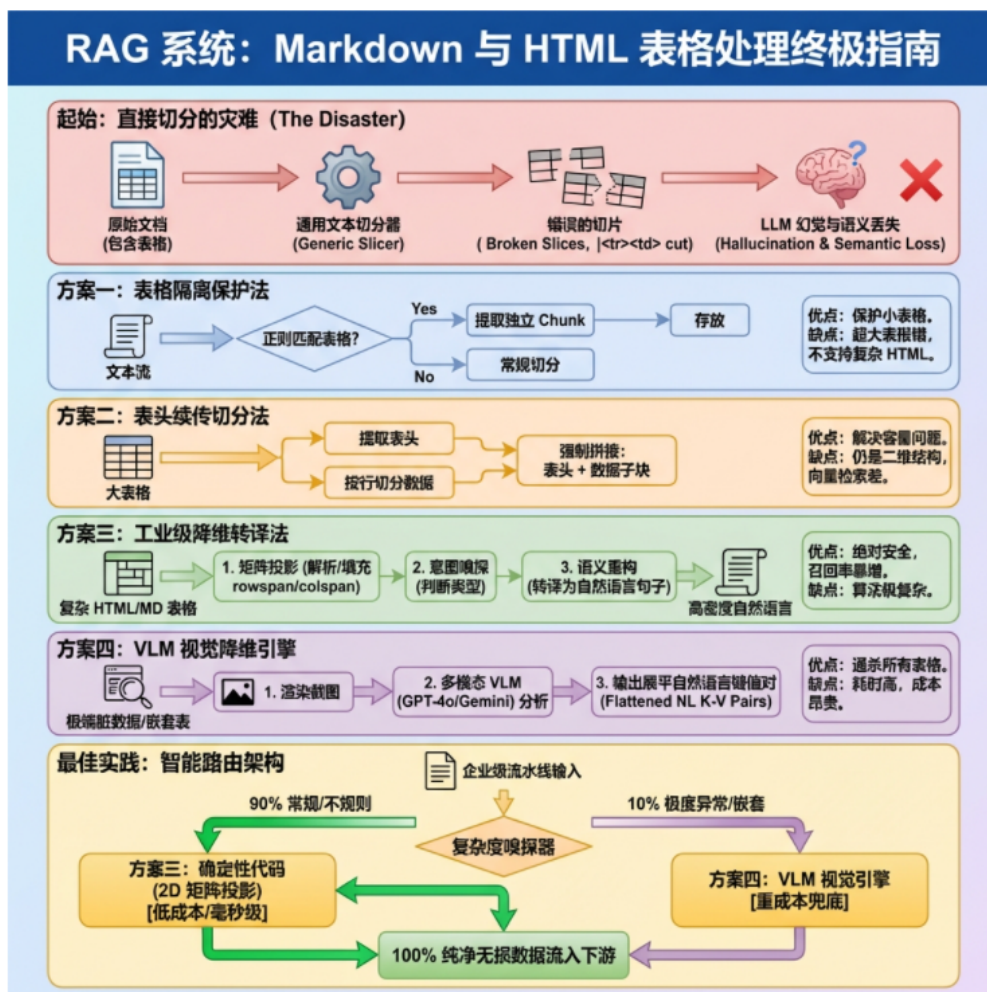
- **优点**：真正的降维打击，通杀世界上所有人类肉眼能看懂的表格。
- **缺点**：推理耗时极长（单图 3-5 秒），API Token 成本高昂。

最佳实践：智能路由架构

在实际的企业级流水线中，**单一方案永远不是最优解**。最成熟的架构是 **方案三 + 方案四 4 的混合动态路由**：

用极低成本、毫秒级响应的**确定性代码（2D 矩阵投影）**去处理 90% 的常规与不规则合并表格；在算法中埋入“复杂度嗅探器”，一旦检测到极度异常的嵌套结构，则立刻路由给 **VLM 视觉引擎** 进行重成本兜底。

这才是确保流入下游的数据做到 100% 纯净无损。



1.5.2 参考实现

```
1  import re
2  from typing import List
3  from bs4 import BeautifulSoup
4
5
6  class MarkdownTableLinearizer:
7      """
8      解决: HTML复杂合并单元格、无表头KV表、左上角空置交叉表、原生MD表
9      """
10
11     HTML_TABLE_PATTERN = re.compile(r"<table.*?>.*?</table>", re.IGNORECASE |
re.DOTALL)
12     MD_TABLE_PATTERN = re.compile(
13         r'((?:^[\t]*\|.*\|[\t]*\n)'
14         r'?(?:^[\t]*\|[\t]*[-:]+[-| :]*\|[\t]*\n)'
15         r'?(?:^[\t]*\|.*\|[\t]*\n(?:\n|$))*)',
16         re.MULTILINE
17     )
18
19     @classmethod
20     def process(cls, content: str) -> str:
21         if not content:
22             return content
23
24         if "<table" in content.lower():
25             content = cls.HTML_TABLE_PATTERN.sub(cls._replace_html_table,
content)
26
27         if "|" in content:
28             content = cls.MD_TABLE_PATTERN.sub(cls._replace_md_table, content)
29
30         return content
31
32     @classmethod
33     def _replace_html_table(cls, match) -> str:
34         html_content = match.group(0)
35         soup = BeautifulSoup(html_content, "html.parser")
36         table = soup.find("table")
37         if not table: return html_content
38
39         rows = table.find_all("tr")
40         if not rows: return html_content
41
42         # 嗅探 HTML 中是否使用了标准的 <th> 表头标签
43         has_th = len(table.find_all("th")) > 0
44
45         grid = []
46         for _ in range(len(rows)):
47             grid.append([])
48
49         for row_idx, row in enumerate(rows):
50             col_idx = 0
51             for cell in row.find_all(['td', 'th']):
52                 while col_idx < len(grid[row_idx]) and grid[row_idx][col_idx] is
not None:
53                     col_idx += 1
```

```

54
55         rowspan = int(cell.get('rowspan', 1))
56         colspan = int(cell.get('colspan', 1))
57         # 使用 separator 保证 <br> 换行能变成空格，不会粘连
58         text = cell.get_text(separator=" ", strip=True)
59
60         for r in range(row_idx, row_idx + rowspan):
61             while len(grid) <= r: grid.append([])
62             while len(grid[r]) < col_idx + colspan: grid[r].append(None)
63             for c in range(col_idx, col_idx + colspan):
64                 grid[r][c] = text
65             col_idx += colspan
66
67         return cls._grid_to_text(grid, is_md=False, has_th=has_th)
68
69     @classmethod
70     def _replace_md_table(cls, match) -> str:
71         md_text = match.group(0).strip()
72         lines = md_text.split('\n')
73         grid = []
74         for line in lines:
75             if re.match(r'^[ \t]*\|[\ \t\-\:]+\|[\ \t]*$', line): continue
76             cells = [cell.strip() for cell in line.strip('|').split('|')]
77             grid.append(cells)
78         # Markdown 表格天生自带表头结构
79         return cls._grid_to_text(grid, is_md=True, has_th=False)
80
81     @classmethod
82     def _grid_to_text(cls, grid: List[List[str]], is_md: bool, has_th: bool) ->
str:
83
84         if not grid or not grid[0]: return ""
85
86         cols_count = max(len(r) for r in grid)
87         # 补齐不规则行的列数，防越界
88         for r in grid:
89             while len(r) < cols_count: r.append("")
90
91         is_header_row = False
92         if is_md or has_th:
93             is_header_row = True
94         # 重点防御：如果左上角是空的，这绝对是一个交叉表头！绝不能当废数据丢弃！
95         elif grid[0][0] == "":
96             is_header_row = True
97         # 一般大于两列的表格，第一行基本都是表头
98         elif cols_count > 2:
99             is_header_row = True
100
101         res = []
102
103         if not is_header_row and cols_count == 2:
104             # 【策略 1：纯 K-V 表格】
105             for r in grid:
106                 k, v = r[0], r[1]
107                 if k or v:
108                     k_str = k if k else "未知属性"
109                     v_str = v if v else "无"
110                     res.append(f"- 【{k_str}】: {v_str}。")
111         else:

```

```

111         # 【策略 2：带有表头定义的标准/交叉表格】
112         headers = grid[0]
113         for r in grid[1:]:
114             # 跳过完全空的数据行
115             if not any(r): continue
116
117             subject = r[0] if r[0] else "未知项目"
118             subject_header = headers[0] if headers[0] else ""
119
120             props = []
121             for c in range(1, cols_count):
122                 head = headers[c] if headers[c] else f"属性{c}"
123                 val = r[c] if r[c] else ""
124
125                 if val and val not in ('-', '/', '\\', '无'):
126                     props.append(f"{head}为{val}")
127
128             if props:
129                 prop_str = ", ".join(props)
130                 if subject_header:
131                     res.append(f"- 【{subject}】(对应{subject_header}): {prop_str}。")
132             else:
133                 # 针对左上角为空的情况，隐藏对应关系描述
134                 res.append(f"- 【{subject}】: {prop_str}。")
135             else:
136                 if subject != "未知项目":
137                     res.append(f"- 【{subject}】")
138
139         return "\n\n" + "\n".join(res) + "\n\n"

```

1.5.3 切分方案调整

1、核心改动点

- 在 `_flush()` 中直接调用表格降维

```

1  def _flush():
2      body = "\n".join(body_lines).strip()
3      if current_title or body:
4          # ===== 表格降维处理 =====
5          body = MarkdownTableLinearizer.process(body)
6
7          # ... 后续处理

```

- 简化切分逻辑

由于表格已经被降维为文本，不再需要：

- `_is_html_table_block()` 函数（保留但不使用）
- 表格独立成块的逻辑
- `has_table` 标记
- 无标题文档也处理

```

1  if not has_title:
2      body = MarkdownTableLinearizer.process(content)
3      sections = [{...}]

```

2、优势

1. **代码更简洁**：去掉了表格检测、表格独立成块、表格合并等复杂逻辑
2. **处理更彻底**：HTML 表格和 Markdown 表格都被统一降维为文本
3. **无碎片化**：不再产生纯表格 chunk，所有内容都是可读文本
4. **时机最优**：在一级切分时就完成降维，后续只需处理纯文本

3、效果对比

之前：

```
1  {
2    "title": "## 收支总表-table",
3    "content": "## 收支总表-table\n\n<table><tr>...</tr></table>",
4    "has_table": true
5  }
```

现在：

```
1  {
2    "title": "## 收支总表",
3    "content": "## 收支总表\n\n- 【本年收入合计】：92,006.77万元。 - 【本年支出合计】：
147,226.08万元。"
4  }
```

4、优化如下：

```
1  import logging
2  import os
3  from typing import List, Tuple
4  import re
5  from langchain_text_splitters import RecursiveCharacterTextSplitter
6  import json
7  from markdown_utils import MarkdownTableLinearizer
8
9  logging.basicConfig(
10     level=logging.DEBUG,
11     format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
12 )
13 logger = logging.getLogger("mineru")
14
15 # ===== 文档处理配置 =====
16 max_content_length: int = 2000 # 切片最大长度
17 min_content_length: int = 500 # 合并短内容的最小长度
18 overlap_sentences: int = 1 # 句子级切分时的重叠句数
19 item_name_chunk_k: int = 3 # 商品名识别时使用的切片数量
20
21 # ----- #
22 #                               Step 1: 获取输入                               #
23 # ----- #
24 def _get_inputs(state):
25     logger.info("step_1:获取输入")
26
27     content = state.get("md_content", "")
28     if content:
29         content = content.replace("\r\n", "\n").replace("\r", "\n")
30
31     file_title = state.get("file_title", "")
```

```

32
33     return content, file_title, max_content_length
34
35 # ----- #
36 #           Step 2: 按标题一级切分 (带层级追踪)           #
37 # ----- #
38 def _split_by_headings(content: str, file_title: str) -> Tuple[List[dict],
bool]:
39     """
40     按 Markdown 标题行切分, 向上追踪层级找 parent_title
41     """
42     logger.info("step_2: 按标题切分并追踪层级")
43
44     heading_re = re.compile(r"^\s*({1,6})\s+(.+)")
45     lines = content.split("\n")
46
47     sections: List[dict] = []
48     current_title = ""
49     current_level = 0
50     body_lines: List[str] = []
51     has_title = False
52     in_fence = False
53
54     # 记录 1-6 级标题的最新足迹
55     hierarchy = [""] * 7
56
57     def _flush():
58         body = "\n".join(body_lines).strip()
59         if current_title or body:
60             # ===== 表格降维处理 =====
61             body = MarkdownTableLinearizer.process(body)
62
63             # 向上找最近的非空父标题
64             parent_title = next(
65                 (hierarchy[lvl] for lvl in range(current_level - 1, 0, -1) if
hierarchy[lvl]),
66                 current_title or file_title
67             )
68
69             sections.append({
70                 "title": current_title,
71                 "body": body,
72                 "file_title": file_title,
73                 "parent_title": parent_title,
74                 "level": current_level,
75             })
76
77     for line in lines:
78         # 代码围栏保护
79         if line.strip().startswith("```") or line.strip().startswith("~~~"):
80             in_fence = not in_fence
81
82         match = heading_re.match(line) if not in_fence else None
83
84         if match:
85             has_title = True
86             _flush()
87

```

```

88         level = len(match.group(1))
89         current_level = level
90         current_title = line.strip()
91         hierarchy[level] = current_title
92
93         # 清除当前标题下的所有子标题足迹
94         for i in range(level + 1, 7):
95             hierarchy[i] = ""
96
97         body_lines = []
98     else:
99         body_lines.append(line)
100
101     _flush()
102     return sections, has_title
103
104     # ----- #
105     #                               Step 4: 二次切分 + 合并短章节                               #
106     # ----- #
107     def _split_and_merge(
108         sections: List[dict],
109         max_length: int,
110         min_length: int,
111     ) -> List[dict]:
112         logger.info("step_4: 二次切分和合并")
113
114         if max_length <= 0:
115             return sections
116
117         # 切分超长章节
118         split_result: List[dict] = []
119         for section in sections:
120             split_result.extend(_split_long_section(section, max_length))
121
122         # 合并过短章节
123         return _merge_short_sections(split_result, min_length)
124
125     def _split_long_section(section: dict, max_length: int) -> List[dict]:
126         """
127         切分超长章节
128         """
129         title = section.get("title", "")
130         body = section.get("body", "")
131         file_title = section.get("file_title", "")
132         parent_title = section.get("parent_title", title)
133
134         # 不超长, 直接返回
135         if len(body) + len(title) <= max_length:
136             return [section]
137
138         # 纯文本切分
139         return _split_text_block(body, title, file_title, parent_title, max_length)
140
141     def _split_text_block(
142         text: str,
143         title: str,
144         file_title: str,
145         parent_title: str,

```

```

146     max_length: int,
147     start_part: int = 0
148 ) -> List[dict]:
149     """
150     切分纯文本块
151     """
152     title_prefix = f"{title}\n\n" if title else ""
153     available = max_length - len(title_prefix)
154
155     # 不超长或无法切分，直接返回
156     if len(text) <= available or available <= 0:
157         return [{
158             "title": title if title else "chunk-1",
159             "body": text,
160             "file_title": file_title,
161             "parent_title": parent_title,
162             "part": start_part + 1,
163         }]
164
165     # 使用标准切分器
166     splitter = RecursiveCharacterTextSplitter(
167         chunk_size=available,
168         chunk_overlap=0,
169         separators=["\n\n", "\n", "。", "!", "?", ";", ".", "!", "?", ";", "
"]
170     )
171
172     pieces = splitter.split_text(text)
173
174     chunks = []
175     for i, piece in enumerate(pieces):
176         chunks.append({
177             "title": f"{title}-{i + 1}" if title else f"chunk-{start_part + i +
178             1}",
179             "body": piece.strip(),
180             "file_title": file_title,
181             "parent_title": parent_title,
182             "part": start_part + i + 1,
183         })
184
185     return chunks
186
187 def _should_merge(current: dict, next_sec: dict, min_length: int) -> bool:
188     """判断是否应该合并两个块"""
189     # 不在同一父标题下，不合并
190     if current.get("parent_title") != next_sec.get("parent_title"):
191         return False
192
193     # 当前块长度足够，不合并
194     if len(current.get("body", "")) >= min_length:
195         return False
196
197     return True
198
199 def _merge_short_sections(sections: List[dict], min_length: int) -> List[dict]:
200     """
201     合并过短的相邻章节
202     """

```

```

202     if not sections:
203         return []
204
205     merged = []
206     current = sections[0]
207
208     for next_sec in sections[1:]:
209         if _should_merge(current, next_sec, min_length):
210             # 合并
211             current["body"] = f"{current['body']}\n\n{next_sec['body']}".strip()
212             # 保留更有意义的标题
213             if next_sec.get("title") and "#" not in next_sec["title"]:
214                 current["title"] = next_sec["title"]
215             else:
216                 merged.append(current)
217                 current = next_sec
218
219     merged.append(current)
220     return merged
221
222     # ----- #
223     #                               Step 5: 组装最终 content                               #
224     # ----- #
225     def _assemble_content(sections: List[dict]) -> List[dict]:
226         """
227         将 title + body 组装为最终的 content 字段
228         """
229         logger.info("step_5: 组装 content")
230
231         result: List[dict] = []
232         for sec in sections:
233             title = sec.get("title", "")
234             body = sec.get("body", "")
235
236             content = f"{title}\n\n{body}".strip() if title and body else (title or
body)
237
238             chunk = {
239                 "title": title,
240                 "content": content,
241                 "file_title": sec.get("file_title", ""),
242             }
243
244             # 保留溯源信息
245             if "parent_title" in sec:
246                 chunk["parent_title"] = sec["parent_title"]
247             if "part" in sec:
248                 chunk["part"] = sec["part"]
249
250             result.append(chunk)
251
252         return result
253
254     # ----- #
255     #                               日志 & 备份                               #
256     # ----- #
257     def _log_summary(raw_content: str, sections: List[dict], max_length: int):
258         logger.info("step_6: 日志与输出统计")

```

```

259
260     lines_count = raw_content.count("\n") + 1
261     logger.info(f"原文档行数: {lines_count}")
262     logger.info(f"最终切分章节数: {len(sections)}")
263     logger.info(f"最大切片长度: {max_length}")
264
265     if sections:
266         logger.info("章节预览:")
267         for i, sec in enumerate(sections[:5]):
268             title = sec.get("title", "")[:50]
269             logger.info(f"  {i + 1}. {title}...")
270         if len(sections) > 5:
271             logger.info(f"    ... 还有 {len(sections) - 5} 个章节")
272
273 def _backup_chunks(state, sections: List[dict]):
274     logger.info("step_7: 备份切片")
275
276     local_dir = state.get("file_dir", state.get("local_dir", ""))
277     if not local_dir:
278         logger.debug("未设置 file_dir/local_dir, 跳过备份")
279         return
280
281     try:
282         os.makedirs(local_dir, exist_ok=True)
283         output_path = os.path.join(local_dir, "chunks.json")
284         with open(output_path, "w", encoding="utf-8") as f:
285             json.dump(sections, f, ensure_ascii=False, indent=2)
286         logger.info(f"已备份到: {output_path}")
287     except Exception as e:
288         logger.warning(f"备份失败: {e}")
289
290
291 # ===== 主流程 =====
292 def process_markdown(state: dict) -> dict:
293     """
294     完整的 Markdown 文档处理流程
295
296     Args:
297         state: 包含 md_content, file_title 的字典
298
299     Returns:
300         更新后的 state, 包含 chunks 字段
301     """
302     # Step 1: 获取输入
303     content, file_title, max_length = _get_inputs(state)
304
305     # Step 2: 按标题切分 (内部已包含表格降维)
306     sections, has_title = _split_by_headings(content, file_title)
307
308     # Step 3: 处理全文无标题的情况
309     if not has_title:
310         # 无标题时也要做表格降维
311         body = MarkdownTableLinearizer.process(content)
312         sections = [{
313             "title": "无标题",
314             "body": body,
315             "file_title": file_title,
316             "parent_title": file_title

```

```

317         }]
318         logger.info("全文无标题，作为单个 chunk 处理")
319
320     # Step 4: 二次切分 + 合并
321     sections = _split_and_merge(sections, max_length, min_content_length)
322
323     # Step 5: 组装最终 content
324     sections = _assemble_content(sections)
325
326     # Step 6: 日志统计
327     _log_summary(content, sections, max_length)
328
329     # Step 7: 备份
330     state["chunks"] = sections
331     _backup_chunks(state, sections)
332
333     return state
334
335
336 if __name__ == "__main__":
337     sample_document_path = r"result\sample\auto\sample_new.md"
338
339     with open(sample_document_path, 'r', encoding='utf-8') as f:
340         content = f.read().strip()
341
342     state = {
343         "file_title": "sample_new",
344         "md_content": content,
345         "file_dir": os.path.dirname(sample_document_path)
346     }
347
348     # 执行完整流程
349     state = process_markdown(state)
350
351     print("\n" + "=" * 50)
352     print("切片执行完毕，最终状态字典概览：")
353     print("=" * 50)
354
355     preview_chunks = state.get("chunks", [])[:10]
356     print(json.dumps(preview_chunks, ensure_ascii=False, indent=4))
357     print(f"\n..... (共生成 {len(state.get('chunks', []))} 个 Chunks，详情请查看\nchunks.json)")

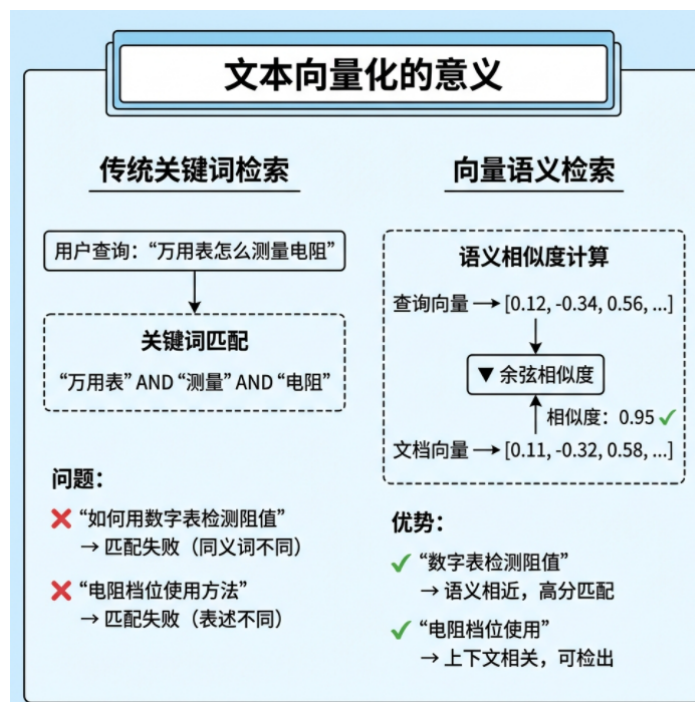
```

二、向量存储

2.1 向量化相关技术

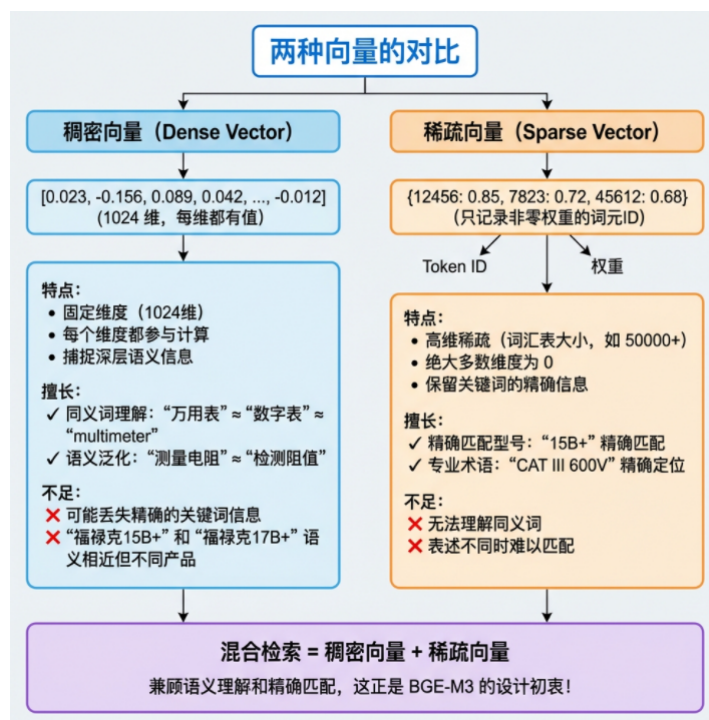
2.1.1 为什么切片需要向量化？

向量化是将文本转换为数学表示的过程，是实现语义检索的基础：



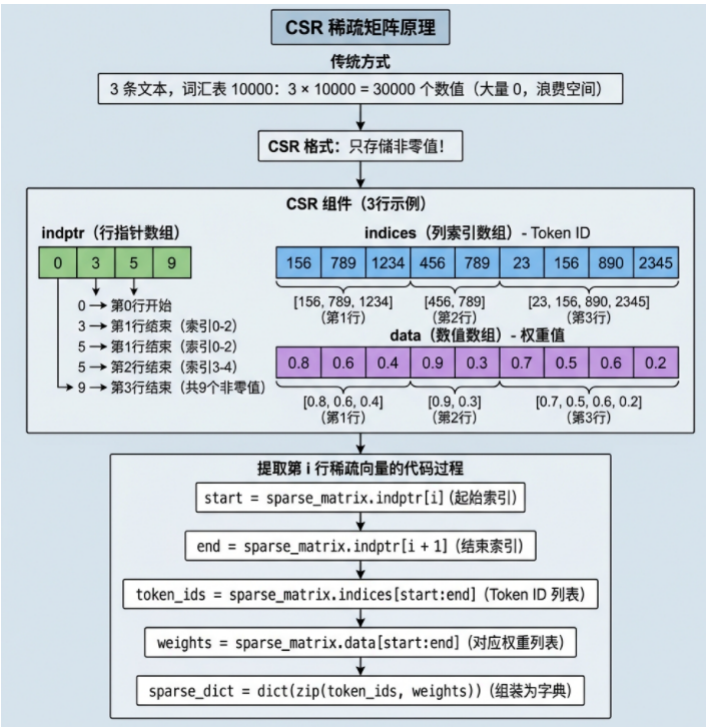
2.1.2 稠密向量 vs 稀疏向量

BGE-M3 模型同时输出两种向量，各有优势：



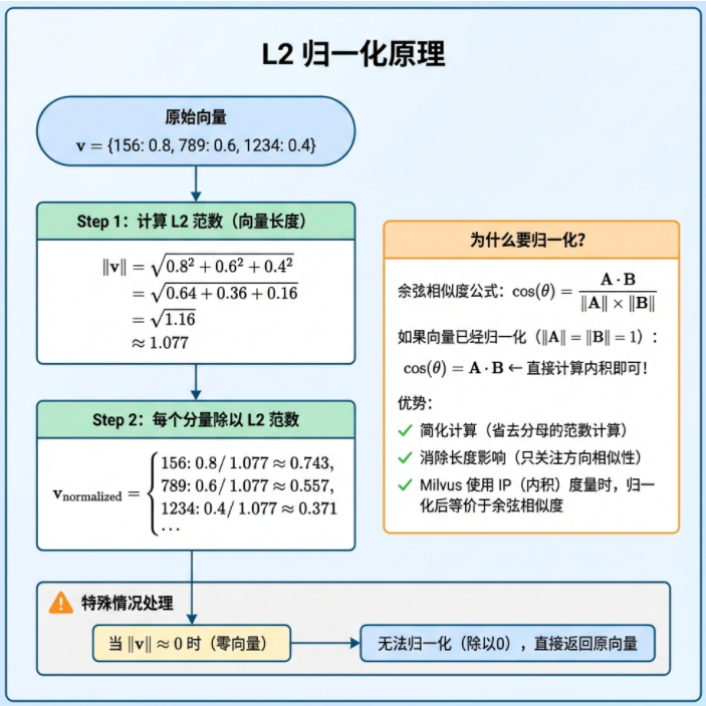
2.1.3 CSR 稀疏矩阵格式

BGE-M3 返回的稀疏向量采用 CSR (Compressed Sparse Row) 格式存储：



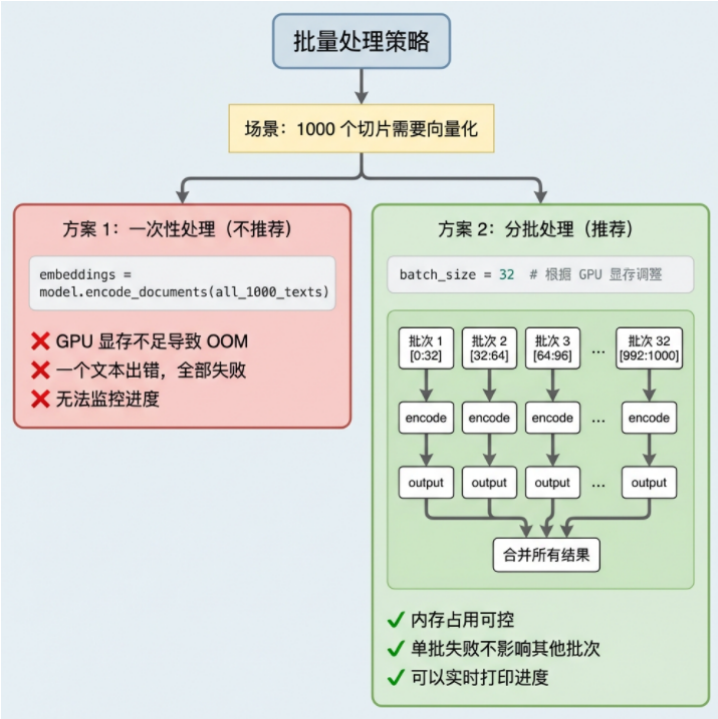
2.1.4 L2 归一化

L2 归一化将向量"拉伸"或"压缩"到单位长度，使得余弦相似度计算更准确:



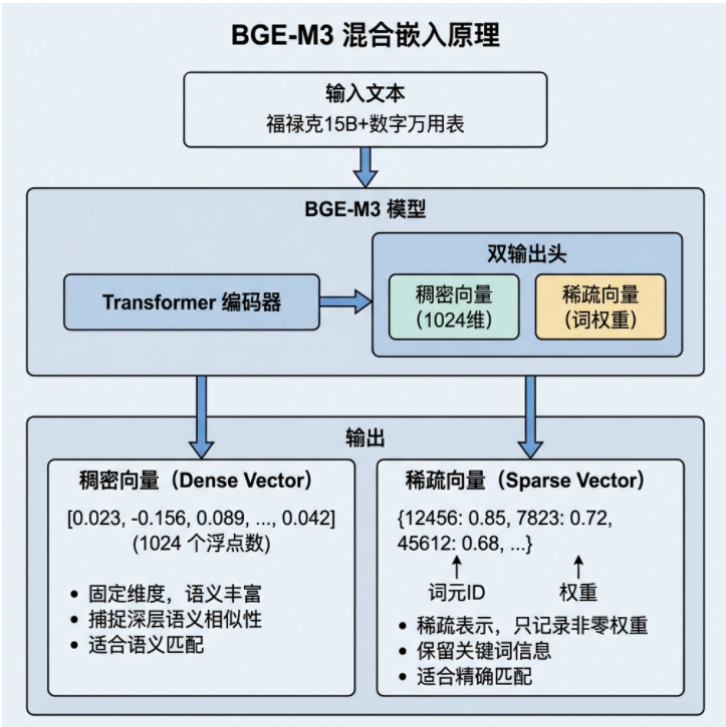
2.1.5 批量处理策略

大规模向量化需要分批处理，以平衡内存占用和处理效率:



2.2 BGE-M3 混合嵌入

BGE-M3 是一个强大的嵌入模型，能同时生成稠密向量和稀疏向量：



为什么需要混合嵌入？

向量类型	优势	劣势	适用场景
稠密向量	语义理解强，能识别同义词	可能丢失关键词信息	"如何使用万用表" ↔ "万用表操作方法"
稀疏向量	精确匹配关键词	语义理解弱	"福禄克15B+" 精确匹配型号
混合检索	兼顾语义和关键词	计算成本略高	生产环境推荐方案

代码示例：

BGEM3EmbeddingFunction 是 PyMilvus（Milvus 向量数据库的 Python SDK）中提供的一个专门用于调用 BGE-M3 模型的 Embedding（向量化）工具类。

它的核心作用是**将文本转换为多种粒度的向量表示（稠密向量、稀疏向量、多向量），从而完美支持 Milvus 的混合检索（Hybrid Search）和多向量检索（Multi-Vector Search），大幅提升检索（尤其是 RAG 场景）的准确率**

```
1  import logging
2  logging.getLogger(__name__).setLevel(logging.ERROR)
3  from pymilvus.model.hybrid import BGEM3EmbeddingFunction
4
5  # 1. 加载模型
6  bge_m3 = BGEM3EmbeddingFunction(
7      model_name="./model/bge-m3",
8      use_fp16=True,
9  )
10
11 # 2. 生成嵌入
12 embeddings = bge_m3.encode_documents(["福禄克15B+数字万用表"])
13
14 # 3. 提取向量
15 dense_vector = embeddings["dense"][0].tolist() # List[float], 长度 1024
16 sparse_matrix = embeddings["sparse"]          # CSR 稀疏矩阵
17
18 # 4. 从 CSR 矩阵提取稀疏向量
19 start_idx = sparse_matrix.indptr[0]
20 end_idx = sparse_matrix.indptr[1]
21 token_ids = sparse_matrix.indices[start_idx:end_idx].tolist()
22 weights = sparse_matrix.data[start_idx:end_idx].tolist()
23 sparse_vector = dict(zip(token_ids, weights)) # Dict[int, float]
24 print(sparse_vector)
```

创建文件 `bge_m3_embedding_util.py` 将其封装成工具：

```
1  import os, logging
2  from typing import Optional, List
3
4  logger = logging.getLogger(__name__)
5  logging.basicConfig(level=logging.INFO)
6  from dotenv import load_dotenv
7  from pymilvus.model.hybrid import BGEM3EmbeddingFunction
8
```

```

9     load_dotenv()
10
11     bge_m3_ef: Optional[BGEM3EmbeddingFunction] = None
12
13
14     def get_beg_m3_embedding_model():
15         global bge_m3_ef
16
17         # 1. 判断
18         if bge_m3_ef is not None:
19             return bge_m3_ef
20
21         # 2. 获取参数
22         model_name = os.getenv('BGE_M3_PATH', './model/bge-m3')
23         device = os.getenv('BGE_DEVICE', 'cpu')
24         use_fp16_str = os.getenv('BGE_FP16', 'False')
25         use_fp16 = use_fp16_str.lower() in ('true', '1', 'yes')
26         try:
27             # 3. 定义嵌入模型对象 # 默认维度1024
28             bge_m3_ef = BGEM3EmbeddingFunction(
29                 model_name=model_name,
30                 device=device,
31                 use_fp16=use_fp16
32             )
33         except Exception as e:
34             return None
35
36         # 4. 返回
37         return bge_m3_ef
38     def generate_hybrid_embeddings(embedding_documents: List[str]):
39         """
40         为文本生成向量嵌入
41         :param embedding_model: 嵌入模型(这里使用BGEM3)
42         :param embedding_documents: 要生成嵌入的文本列表
43         :return: 包含dense和sparse向量的字典
44         """
45         try:
46             # 1. 生成嵌入
47             model = get_beg_m3_embedding_model()
48             embedding_result = model.encode_documents(embedding_documents)
49
50             processed_sparse_result = []
51             # 2. 遍历每一个文档
52             for index in range(len(embedding_documents)):
53                 # 2.1 解构csr矩阵&获取稀疏向量
54                 csr_array = embedding_result['sparse']
55                 # a) 行索引
56                 ind_ptr = csr_array.indptr
57
58                 # b) 获取行索引的起始值
59                 start_ind_ptr = ind_ptr[index]
60                 end_ind_ptr = ind_ptr[index + 1]
61
62                 # c) 获取token_id
63                 token_id = csr_array.indices[start_ind_ptr:end_ind_ptr].tolist()
64
65                 # d) 获取权重
66                 weight = csr_array.data[start_ind_ptr:end_ind_ptr].tolist()

```

```

67
68         # 2.2 获取稀疏向量
69         sparse_vector = dict(zip(token_id, weight))
70
71         processed_sparse_result.append(sparse_vector)
72
73     # 3. 返回
74     return {
75         "dense": [den.tolist() for den in embedding_result["dense"]],
76         "sparse": processed_sparse_result
77     }
78     except Exception as e:
79         return None
80
81 if __name__ == '__main__':
82     dict = generate_hybrid_embeddings(embedding_documents=["我是中国人", "我喜欢自由"])
83     print(dict)

```

接下来详细解释具体向量化过程：

Step 1: 数据校验与标准化预处理

统一输入数据格式，提取并重组文档的层级元数据（Metadata），确保下游处理的一致性。

输入：

- `chunks`：原始切片列表（可能包含 LangChain `Document` 对象、普通字典，或嵌套的列表）。
- `default_filename`：默认文件名（当 metadata 中缺失时使用）。

处理逻辑：

1. **类型兼容与递归**：检查 `chunks` 是否为列表，若是则递归遍历处理每个元素；若是单个对象，则判断是 `Document` 还是 `dict`。
2. **层级标题提取**：遍历 metadata，筛选出所有以 "Header " 开头的键，并按字母/数字顺序排序。
3. 智能字段分配：
 - `title`：取排序后的最后一个 Header（当前块的最细粒度标题）。
 - `parent_title`：将除最后一个之外的所有 Header 用空格拼接，提供上下文。
 - `file_title`：优先取 metadata 中的文件名，否则使用 `default_filename`。

输出：

- 标准化后的字典列表，每个字典包含 `file_title`, `title`, `parent_title`, `content`。

```

1  def parse_chunk(chunk: Any, default_filename: str = "unknown_file") ->
    Union[Dict[str, Any], List[Dict[str, Any]]]:
2      # 1. 兼容列表递归处理
3      if isinstance(chunk, list):
4          return [parse_chunk(c, default_filename) for c in chunk]
5
6      # 2. 兼容对象与字典解析
7      if hasattr(chunk, 'metadata') and hasattr(chunk, 'page_content'):
8          metadata, content = chunk.metadata, chunk.page_content
9      elif isinstance(chunk, dict):
10         metadata, content = chunk.get("metadata", {}), chunk.get("page_content",
        "")
11     else:

```

```

12         raise ValueError(f"不支持的 chunk 类型: {type(chunk)}")
13
14     # 3. 提取并排序 Header
15     headers = [str(metadata[key]).strip() for key in sorted(metadata.keys()) if
key.startswith("Header ")]
16
17     # 4. 智能分配标题层级
18     title = headers[-1] if headers else ""
19     parent_title = " ".join(headers[:-1]) if len(headers) > 1 else ""
20
21     return {
22         "file_title": str(default_filename).strip(),
23         "title": title,
24         "parent_title": parent_title,
25         "content": str(content).strip(),
26     }

```

Step 2: 初始化 BGE-M3 嵌入模型

获取或初始化 BGE-M3 混合嵌入模型实例，确保模型加载成功并准备好进行推理。

输入：

- 无直接输入，依赖底层 `get_beg_m3_embedding_model()` 函数的环境配置。

处理逻辑：

1. 调用封装好的工具函数获取模型实例（通常内部实现了单例模式或缓存机制，避免重复加载消耗显存）。
2. 模型会根据预设的环境变量或默认配置，自动加载到指定设备（如 `cuda` 或 `cpu`），并决定是否启用 FP16 精度加速。
3. 使用 `try-except` 块捕获加载过程中的任何异常（如模型文件缺失、显存不足等）。

输出：

- 初始化成功的 `bge_m3_ef` (BGEM3EmbeddingFunction) 实例。

```

1  # Step 2: 初始化 BGE-M3
2  try:
3      bge_m3_ef = get_beg_m3_embedding_model()
4      logger.info("BGE-M3 模型初始化成功")
5  except Exception as e:
6      raise Exception(f"初始化 BGE-M3 失败: {e}")

```

Step 3: 文本构造与批量向量化生成

按照配置的批次大小，智能拼接文本并调用模型生成混合向量，防止内存溢出（OOM）。

输入：

- `bge_m3_ef`: 初始化好的模型实例。
- `parsed_chunks`: 标准化后的切片列表。
- `batch_size`: 每批处理的切片数量（例如配置为 5）。

处理逻辑：

1. **智能文本拼接**：遍历批次内的每个文档，若 `title` 和 `content` 均存在，则拼接为 `title\ncontent`；若仅有其一，则单独使用，避免产生无意义的前置换行符干扰模型注意力。
2. **批量推理**：调用 `bge_m3_ef.encode_documents(texts)`，一次性生成当前批次的稠密（dense）和稀疏（sparse）向量。

3. **异常降级**：若某批次生成失败，记录错误日志并返回原始数据，防止整个流程崩溃。

输出：

- 包含原始字段及新生成向量字段的字典列表。

```
1 def _process_batch(bge_m3_ef, batch: List[Dict[str, Any]], start_idx: int, total:
  int) -> List[Dict[str, Any]]:
2     try:
3         # 1. 智能构造输入文本
4         texts = []
5         for doc in batch:
6             title, content = doc.get("title", "").strip(), doc.get("content",
  "").strip()
7             if title and content: texts.append(f"{title}\n{content}")
8             elif content: texts.append(content)
9             else: texts.append(title)
10
11         # 2. 批量生成向量
12         embeddings = bge_m3_ef.encode_documents(texts)
13         if not embeddings or "dense" not in embeddings:
14             logger.warning(f"批次 {start_idx + 1} 未能生成向量")
15             return batch
16
17         # ... (接 Step 4 的解析逻辑)
```

Step 4: 向量解析与安全序列化转换

从模型返回的复杂数据结构中提取向量，并进行数学归一化和类型安全转换，为 JSON 存储做准备。

输入：

- **embeddings**：模型返回的字典（包含 **dense** 列表和 **sparse** SciPy CSR 矩阵）。
- **batch**：当前批次的原始数据。

处理逻辑：

1. **稠密向量提取**：获取 `embeddings["dense"][j]`，并使用 `.tolist()` 将其从 NumPy 数组转为 Python 原生列表。
2. **稀疏向量解析**：利用 SciPy CSR 矩阵的 `indptr`（行指针）和 `indices`（列索引），精准切片提取当前文档的 `token_ids` 和 `weights`。
3. **L2 归一化与类型强转（△ 核心避坑）**：调用 `normalize_sparse_vector`，计算非零权重的 L2 范数并进行归一化。**强制将 key 转为 int，value 转为原生 float**，防止后续 `json.dump` 时因 NumPy 类型报错。

输出：

- 组装完成的最终 **item** 字典，包含可序列化的 **dense_vector** 和 **sparse_vector**。

```
1 def normalize_sparse_vector(sparse_vec: Dict[int, float]) -> Dict[int, float]:
2     if not sparse_vec: return sparse_vec
3     values = np.array(list(sparse_vec.values()), dtype=np.float64)
4     l2_norm = np.linalg.norm(values)
5     if l2_norm < 1e-9: return {int(k): float(v) for k, v in sparse_vec.items()}
6
7     # 归一化并强制转为 Python 原生类型，确保 JSON 序列化安全
8     return {int(k): float(v) for k, v in zip(sparse_vec.keys(), values /
  l2_norm)}
```

```

10 # 在 _process_batch 循环中调用:
11 sparse_matrix = embeddings.get("sparse")
12 sparse_vector = {}
13 if sparse_matrix is not None:
14     start, end = sparse_matrix.indptr[j], sparse_matrix.indptr[j + 1]
15     token_ids = sparse_matrix.indices[start:end].tolist()
16     weights = sparse_matrix.data[start:end].tolist()
17     sparse_vector = normalize_sparse_vector(dict(zip(token_ids, weights)))

```

Step 5: 结果验证与持久化存储

对向量化后的最终结果进行抽样校验，确保数据质量符合预期，并安全写入磁盘。

输入：

- `result` : 完整处理完毕的切片列表。
- `output_path` : 目标 JSON 文件的保存路径。

处理逻辑：

1. **结构抽样验证**：打印第一个切片的元数据、向量维度、稀疏向量非零元素数量。
2. **归一化数学验证**：抽取 `sparse_vector` 的值计算 L2 范数，验证其结果是否无限接近 `1.0`，确保 Step 4 的归一化逻辑正确执行。
3. **安全持久化**：使用 `json.dump` 写入文件，必须指定 `ensure_ascii=False`（保证中文不乱码）和 `indent=4`（保证文件可读性）。

输出：

- 磁盘上生成的 `chunks_1_vector.json` 文件。

```

1 # 1. 验证输出数据
2 if output_chunks:
3     first_chunk = output_chunks[0]
4     sparse_vec = first_chunk.get('sparse_vector', {})
5     if sparse_vec:
6         values = np.array(list(sparse_vec.values()))
7         l2_norm = np.linalg.norm(values)
8         print(f" - sparse_vector L2范数: {l2_norm:.6f} (归一化后应接近1.0)")
9
10 # 2. 持久化保存
11 with open(output_path, "w", encoding="utf-8") as f:
12     json.dump(result, f, ensure_ascii=False, indent=4)
13
14 print(f"\n✅ 已安全保存到: {output_path}")

```

Step6: 完整过程

```

1 import os
2 import json
3 import logging
4 from typing import List, Union, Dict, Any
5 import numpy as np
6
7 # 假设该文件与当前脚本在同一目录下
8 from bge_m3_embedding_util import get_beg_m3_embedding_model
9
10 logging.basicConfig(
11     level=logging.DEBUG,
12     format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'

```

```

13 )
14 logger = logging.getLogger("生成向量化")
15
16 embedding_batch_size = 5
17
18 from typing import Any, Dict, List, Union
19
20 def parse_chunk(chunk: Any, default_filename: str = "unknown_file") ->
    Union[Dict[str, Any], List[Dict[str, Any]]]:
21     """
22     统一解析 chunk, 兼容单个 LangChain Document 对象、普通字典, 以及包含它们的列表
23     """
24     # 新增: 如果传入的是列表, 自动遍历处理每一个元素
25     if isinstance(chunk, list):
26         return [parse_chunk(c, default_filename) for c in chunk]
27
28     # 1. 解析单个对象
29     if hasattr(chunk, 'metadata') and hasattr(chunk, 'page_content'):
30         # LangChain Document 对象
31         metadata = chunk.metadata
32         content = chunk.page_content
33     elif isinstance(chunk, dict):
34         # 普通字典格式
35         metadata = chunk.get("metadata", {})
36         content = chunk.get("page_content", "")
37     else:
38         raise ValueError(f"不支持的 chunk 类型: {type(chunk)}")
39
40     # 2. 提取所有的 Header, 按数字顺序排序 (Header 1, Header 2, Header 3...)
41     headers = []
42     for key in sorted(metadata.keys()):
43         if key.startswith("Header "):
44             headers.append(str(metadata[key]).strip())
45
46     # 3. 根据规则分配 title 和 parent_title
47     if headers:
48         title = headers[-1]
49         parent_title = " ".join(headers[:-1]) # 除最后一个外, 用空格拼接
50     else:
51         title = ""
52         parent_title = ""
53
54     # 4. 获取 file_title: 优先从 metadata 取, 其次用 Header 1, 最后用默认值
55     file_title = default_filename
56
57     return {
58         "file_title": str(file_title).strip(),
59         "title": title,
60         "parent_title": parent_title,
61         "content": str(content).strip(),
62     }
63
64 def normalize_sparse_vector(sparse_vec: Dict[int, float]) -> Dict[int, float]:
65     """
66     对稀疏向量做 L2 归一化 (仅处理非零维度)
67     强制转换 key 为 int, value 为原生 Python float, 确保 JSON 序列化不报错
68     """
69     if not sparse_vec:

```

```

70         return sparse_vec
71
72     values = np.array(list(sparse_vec.values()), dtype=np.float64)
73     l2_norm = np.linalg.norm(values)
74
75     if l2_norm < 1e-9:
76         return {int(k): float(v) for k, v in sparse_vec.items()}
77
78     normalized_values = values / l2_norm
79     return {int(k): float(v) for k, v in zip(sparse_vec.keys(),
80 normalized_values)}
81
82 def process(chunks: List[Any], default_filename: str = "document") ->
83 List[Dict[str, Any]]:
84     if not isinstance(chunks, list) or not chunks:
85         raise Exception("chunks 为空或无效")
86
87     # Step 1: 统一格式化输入数据
88     parsed_chunks = [parse_chunk(c, default_filename) for c in chunks]
89     logger.info(f"step_1: 开始为 {len(parsed_chunks)} 个切片生成向量")
90
91     # Step 2: 初始化 BGE-M3
92     try:
93         bge_m3_ef = get_bge_m3_embedding_model()
94     except Exception as e:
95         raise Exception(f"初始化 BGE-M3 失败: {e}")
96
97     # Step 3: 批量处理
98     output_data = []
99     batch_size = embedding_batch_size
100
101     for i in range(0, len(parsed_chunks), batch_size):
102         batch = parsed_chunks[i:i + batch_size]
103         batch_output = _process_batch(bge_m3_ef, batch, i, len(parsed_chunks))
104         output_data.extend(batch_output)
105
106     logger.info(f"step_2: 向量化完成, 共 {len(output_data)} 个切片")
107     return output_data
108
109 def _process_batch(bge_m3_ef, batch: List[Dict[str, Any]], start_idx: int,
110 total: int) -> List[Dict[str, Any]]:
111     """处理一个批次的切片"""
112     try:
113         # 构造输入文本: 智能处理 title 为空的情况, 避免多余换行
114         texts = []
115         for doc in batch:
116             title = doc.get("title", "").strip()
117             content = doc.get("content", "").strip()
118             if title and content:
119                 texts.append(f"{title}\n{content}")
120             elif content:
121                 texts.append(content)
122             else:
123                 texts.append(title)
124
125         # 批量生成向量
126         embeddings = bge_m3_ef.encode_documents(texts)

```

```

125         if not embeddings or "dense" not in embeddings:
126             logger.warning(f"批次 {start_idx + 1}-{start_idx + len(batch)} 未能生
成向量")
127             return batch
128
129         output = []
130         sparse_matrix = embeddings.get("sparse")
131
132         for j, doc in enumerate(batch):
133             # 提取稠密向量 (确保转为原生 list)
134             dense_vec_raw = embeddings["dense"][j]
135             dense_vector = dense_vec_raw.tolist() if hasattr(dense_vec_raw,
'tolist') else list(dense_vec_raw)
136
137             # 提取稀疏向量 (pymilvus 返回的是 scipy.sparse 矩阵)
138             sparse_vector = {}
139             if sparse_matrix is not None:
140                 start = sparse_matrix.indptr[j]
141                 end = sparse_matrix.indptr[j + 1]
142                 token_ids = sparse_matrix.indices[start:end].tolist()
143                 weights = sparse_matrix.data[start:end].tolist()
144                 raw_sparse_dict = dict(zip(token_ids, weights))
145                 sparse_vector = normalize_sparse_vector(raw_sparse_dict)
146
147             # 构建最终输出
148             item = {
149                 "content": doc.get("content"),
150                 "title": doc.get("title"),
151                 "parent_title": doc.get("parent_title", ""),
152                 "file_title": doc.get("file_title"),
153                 "dense_vector": dense_vector,
154                 "sparse_vector": sparse_vector
155             }
156             output.append(item)
157
158             logger.info(f"成功处理批次 {start_idx + 1}-{min(start_idx + len(batch),
total)}/{total}")
159         return output
160
161     except Exception as e:
162         logger.error(f"批次 {start_idx + 1}-{start_idx + len(batch)} 处理失败:
{e}", exc_info=True)
163         # 容错: 返回原始数据, 不含向量
164         return batch
165 if __name__ == '__main__':
166     # -----
167     # Step 1: 配置路径
168     # -----
169     temp_dir = r"./result/sample/auto"
170     os.makedirs(temp_dir, exist_ok=True) # 确保输出目录存在
171
172     output_path = os.path.join(temp_dir, "chunks_1_vector.json")
173
174     # -----
175     # Step 2: 读取输入数据
176     # -----
177     # 输入: 上一个商品识别节点处理后的状态
178     # input_path = os.path.join(temp_dir, "chunks_1.json")

```

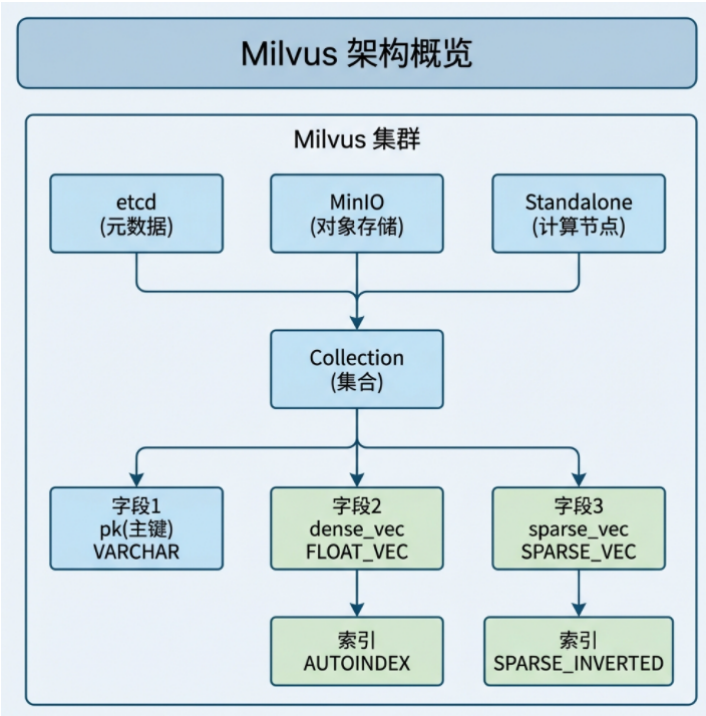
```

179     # print(f"正在读取输入文件：{input_path}")
180
181     # with open(input_path, "r", encoding="utf-8") as f:
182     #     chunks = json.load(f)
183
184     # print(f"读取到 {len(chunks)} 个切片")
185
186     # 注意：请确保在此处加载或定义你的 final_chunks 变量
187     if 'final_chunks' not in locals():
188         logger.warning("未检测到 final_chunks 变量，请确保在运行前已加载数据。")
189         # 这里仅作占位，实际运行时请替换为你的真实数据列表
190         final_chunks = []
191
192     print("开始执行向量化...")
193     # 传入 default_filename 以便在 metadata 缺失文件名时作为后备
194     result = process(final_chunks, default_filename="中国科学院国家天文台2023年部门
预算")
195
196     # -----
197     # Step 2：验证输出数据
198     # -----
199     output_chunks = result
200     print(f"\n处理完成，共 {len(output_chunks)} 个切片")
201
202     if output_chunks:
203         first_chunk = output_chunks[0]
204         print("\n第一个切片数据结构:")
205         print(f" - file_title: {first_chunk.get('file_title')}")
206         print(f" - parent_title: {first_chunk.get('parent_title')}")
207         print(f" - title: {first_chunk.get('title')}")
208         print(f" - content: {first_chunk.get('content', '')[:50]}...")
209
210         dense_vec = first_chunk.get('dense_vector', [])
211         sparse_vec = first_chunk.get('sparse_vector', {})
212
213         print(f"\n向量信息:")
214         print(f" - dense_vector 维度: {len(dense_vec)}")
215         print(f" - dense_vector 前5维: {dense_vec[:5] if dense_vec else '无'}")
216         print(f" - sparse_vector 非零元素数: {len(sparse_vec)}")
217
218         if sparse_vec:
219             sample_items = list(sparse_vec.items())[:3]
220             print(f" - sparse_vector 前3项: {sample_items}")
221
222             # 验证稀疏向量是否已归一化
223             values = np.array(list(sparse_vec.values()))
224             l2_norm = np.linalg.norm(values)
225             print(f" - sparse_vector L2范数: {l2_norm:.6f} (归一化后应接近1.0)")
226
227     # -----
228     # Step 3：保存输出文件
229     # -----
230     with open(output_path, "w", encoding="utf-8") as f:
231         # ensure_ascii=False 保证中文正常显示，indent=4 保证可读性
232         json.dump(result, f, ensure_ascii=False, indent=4)
233
234     print(f"\n已保存到: {output_path}")

```

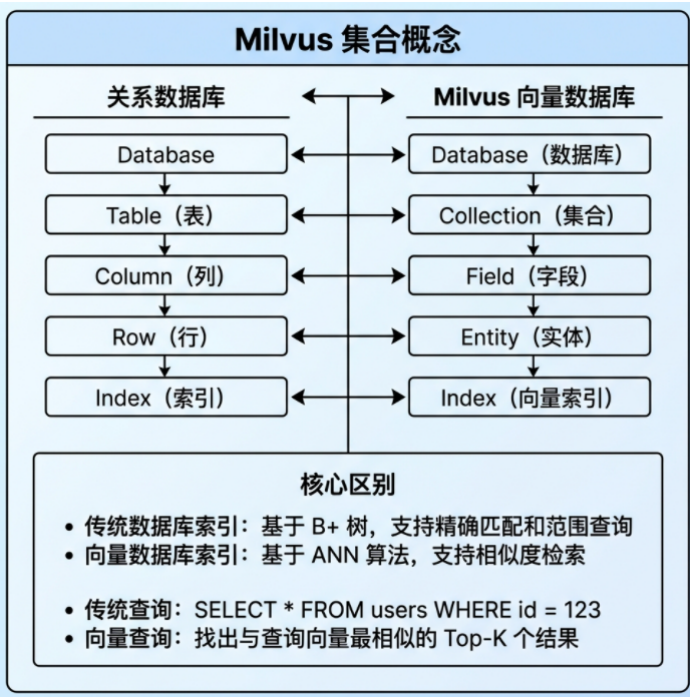
2.3 Milvus 向量数据库

Milvus 是专为向量检索设计的开源数据库：



2.3.1 Milvus 集合 (Collection)

Milvus 中的**集合 (Collection)** 类似于关系数据库中的表，是存储和检索向量数据的基本单位：



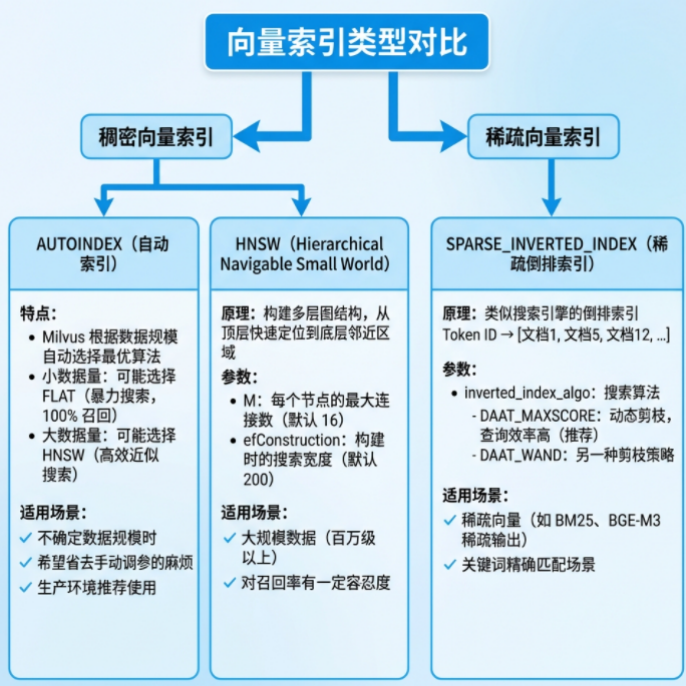
2.3.2 Schema 设计

Schema 定义了集合中的字段结构，包括字段名称、数据类型、约束等：



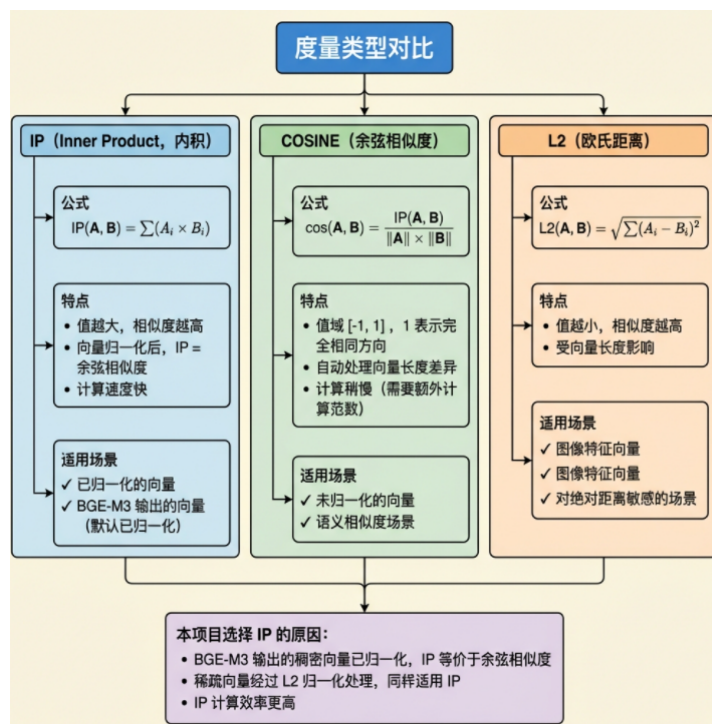
2.3.3 向量索引类型

索引是加速向量检索的关键, 不同索引类型适用于不同场景:



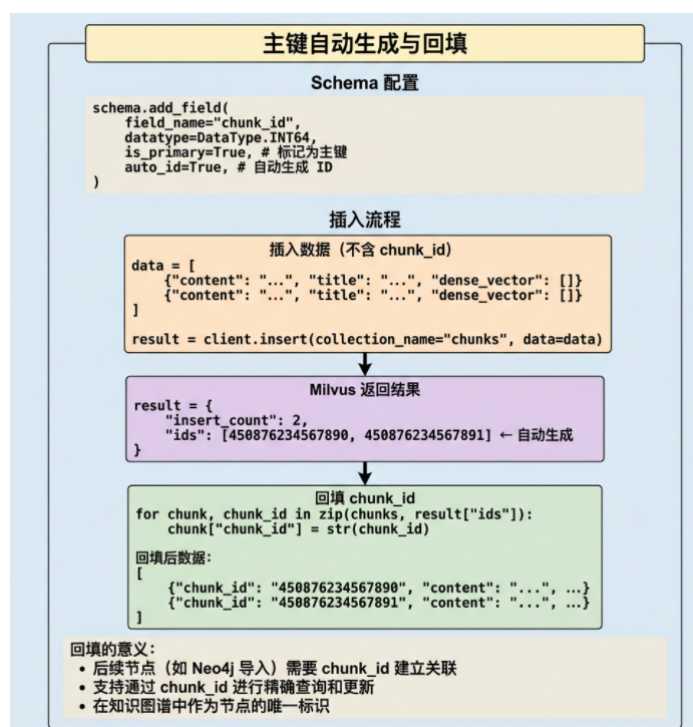
2.3.4 度量类型 (Metric Type)

度量类型决定了如何计算向量间的相似度:



2.3.5 主键自动生成与回填

Milvus 支持自动生成主键, 插入后需要将主键回填到业务数据中:



集合 Schema 设计:

```
1 from pymilvus import DataType
2
3 # 创建 Schema
4 schema = client.create_schema(enable_dynamic_fields=True)
5
6 # 添加字段
7 schema.add_field(
8     field_name="pk",
9     datatype=DataType.VARCHAR,
10    is_primary=True,
11    auto_id=True, # 自动生成主键
```

```

12     max_length=100
13 )
14 schema.add_field(
15     field_name="file_title",
16     datatype=DataType.VARCHAR,
17     max_length=65535
18 )
19 schema.add_field(
20     field_name="content",
21     datatype=DataType.VARCHAR,
22     max_length=65535
23 )
24 schema.add_field(
25     field_name="dense_vector",
26     datatype=DataType.FLOAT_VECTOR,
27     dim=1024          # BGE-M3 输出维度
28 )
29 schema.add_field(
30     field_name="sparse_vector",
31     datatype=DataType.SPARSE_FLOAT_VECTOR
32 )

```

索引配置:

```

1  index_params = client.prepare_index_params()
2
3  # 稠密向量索引（自动选择最优算法）
4  index_params.add_index(
5      field_name="dense_vector",
6      index_name="dense_vector_index",
7      index_type="AUTOINDEX",          # 自动选择 HNSW 等算法
8      metric_type="IP"                 # 内积（余弦相似度）
9  )
10
11 # 稀疏向量索引（倒排索引）
12 index_params.add_index(
13     field_name="sparse_vector",
14     index_name="sparse_inverted_index",
15     index_type="SPARSE_INVERTED_INDEX",
16     metric_type="IP"
17 )

```

2.3.6 构建Milvus工具

创建 `milvus_util.py` 文件。

```

1  import os
2  import logging
3
4  logger = logging.getLogger(__name__)
5  logging.basicConfig(level=logging.INFO)
6
7  from dotenv import load_dotenv
8
9  load_dotenv()
10
11 from typing import Optional
12 from pymilvus import MilvusClient, WeightedRanker, AnnSearchRequest

```

```

13 milvus_client: Optional[MilvusClient] = None
14
15 def get_milvus_client() -> Optional[MilvusClient]:
16     global milvus_client
17
18     # 1.判断
19     if milvus_client is not None:
20         return milvus_client
21
22     # 2. 获取参数
23     try:
24         milvus_uri = os.getenv('MILVUS_URL', 'http://192.168.200.130:19530')
25
26         # 3. 定义MilvusClient对象
27         milvus_client = MilvusClient(
28             uri=milvus_uri
29         )
30
31         return milvus_client
32     except Exception as e:
33         logger.error(f"Milvus客户端创建失败:{str(e)}")
34         return None
35
36     # -----
37     # 混合检索
38     # -----
39
40
41     # -----
42     # 创建混合检索请求
43     # -----
44
45 def create_hybrid_search_requests(dense_vector,
46                                  sparse_vector,
47                                  dense_params=None,
48                                  sparse_params=None,
49                                  expr=None,
50                                  limit=5):
51     """
52     创建混合搜索请求
53
54     :param dense_vector: 稠密向量
55     :param sparse_vector: 稀疏向量
56     :param dense_params: 稠密向量搜索参数，默认为None
57     :param sparse_params: 稀疏向量搜索参数，默认为None
58     :param expr: 查询表达式，默认为None
59     :param limit: 返回结果数量限制，默认为5
60     :return: 包含稠密和稀疏搜索请求的列表
61     """
62     # 默认参数
63     if dense_params is None:
64         dense_params = {"metric_type": "COSINE"}
65     if sparse_params is None:
66         sparse_params = {"metric_type": "IP"}
67
68     # 创建稠密向量搜索请求
69     dense_req = AnnSearchRequest(
70         data=[dense_vector],
71         anns_field="dense_vector",

```

```

71         param=dense_params,
72         expr=expr,
73         limit=limit
74     )
75
76     # 创建稀疏向量搜索请求
77     sparse_req = AnnSearchRequest(
78         data=[sparse_vector],
79         anns_field="sparse_vector",
80         param=sparse_params,
81         expr=expr,
82         limit=limit
83     )
84
85     return [dense_req, sparse_req]
86
87     # -----
88     # 执行混合检索请求
89     # -----
90     def execute_hybrid_search_query(milvus_client: MilvusClient,
91                                     collection_name,
92                                     search_requests,
93                                     ranker_weights=(0.5, 0.5),
94                                     norm_score=False,
95                                     limit=5,
96                                     output_fields=None,
97                                     search_params=None):
98         """
99         执行混合搜索
100         :param collection_name: 集合名称
101         :param search_requests: 搜索请求列表, 通常是[dense_req, sparse_req]
102         :param ranker_weights: 权重排名器的权重, 默认为(0.5, 0.5)
103         :param norm_score: 是否对分数进行归一化, 默认为True
104         :param limit: 返回结果数量限制, 默认为5
105         :param output_fields: 要返回的字段列表, 默认为None
106         :param search_params: 搜索参数, 默认为None
107         :return: 搜索结果
108         """
109         try:
110             # 创建权重融合排序器
111             rerank = WeightedRanker(ranker_weights[0], ranker_weights[1],
112                                     norm_score=norm_score)
113
114             # 默认输出字段
115             if output_fields is None:
116                 output_fields = ["content"]
117
118             # 执行搜索
119             res = milvus_client.hybrid_search(
120                 collection_name=collection_name,
121                 reqs=search_requests,
122                 ranker=rerank,
123                 limit=limit,
124                 output_fields=output_fields,
125                 search_params=search_params
126             )
127
128             # 动态计算所有查询返回的结果总数

```

```

128         total_hits = sum(len(hits) for hits in res) if res else 0
129         logger.info(f"Milvus 混合搜索完成, 共处理 {len(res) if res else 0} 个查询, 总计找到 {total_hits} 个结果")
130         return res
131     except Exception as e:
132         logger.error(f"执行Milvus混合搜索时发生错误: {e}")
133         return None
134 def fetch_chunks_by_chunk_ids(
135     collection_name: str,
136     chunk_ids,
137     *,
138     output_fields=None,
139     batch_size: int = 100,
140 ):
141     """
142     通过 chunk_id (主键) 批量查询切片字段
143     返回: List[dict], 元素为 Milvus entity (字段字典)。
144     """
145     client = get_milvus_client()
146     if not collection_name:
147         return []
148     if output_fields is None:
149         # 默认返回字段需与 collection schema 保持一致
150         output_fields = ["chunk_id", "content", "title", "file_title", "parent_title"]
151
152     results = []
153     # 分批, 避免一次性过大
154     for i in range(0, len(chunk_ids), batch_size):
155         batch = chunk_ids[i : i + batch_size]
156         # get (主键直取)
157         try:
158             got = client.get(collection_name=collection_name, ids=batch, output_fields=output_fields)
159             if got:
160                 results.extend(got)
161             continue
162         except Exception as e:
163             logger.error(f"Milvus get() 查询失败: {e}")
164
165     return results

```

2.4 写入向量库

2.4.1 实现流程

Step1: 获取并校验切片数据

获取切片列表, 验证数据有效性并提取向量维度。

处理逻辑:

1. 获取 "chunks"
2. 如果 chunks 为空, 记录警告并直接返回 (幂等设计)
3. 从第一条切片中提取 dense_vector 的维度
4. 如果向量维度为 0, 抛出异常

输出:

- 验证通过的切片列表 `chunks`
- 向量维度 `vector_dim`

代码片段：

```

1  def process(chunks):
2      # 1. 校验
3      if not chunks:
4          logger.warning("chunks 为空, 跳过导入")
5          return None
6
7      vector_dim = _get_vector_dim(chunks)
8      logger.info(f"step_1:准备导入 {len(chunks)} 条数据, 向量维度: {vector_dim}")
9
10
11 def _get_vector_dim(chunks: List[Dict]) -> int:
12     """从首条 chunk 中获取稠密向量维度。
13
14     Args:
15         chunks: 切片数据列表。
16
17     Returns:
18         向量维度。
19
20     Raises:
21         Exception: 首条 chunk 不包含 dense_vector 时抛出。
22     """
23     dim = len(chunks[0].get("dense_vector", []))
24     if dim == 0:
25         raise Exception(f"切片数据不包含 dense_vector")
26     return dim

```

Step2: 连接 Milvus

获取 Milvus 客户端连接实例。

处理逻辑：

1. 调用 `get_milvus_client()` 获取客户端单例
2. 客户端根据环境变量 `MILVUS_URL` 配置连接地址
3. 首次调用时建立连接, 后续调用复用

环境变量配置：

- `MILVUS_URL` : Milvus 服务地址, 例如 "`http://localhost:19530`"

代码片段：

```

1  try:
2      client = get_milvus_client()
3      collection_name = chunks_collection # 从配置获取集合名

```

Step3: 检查并创建集合

检查目标集合是否存在, 不存在则创建。

处理逻辑：

1. 调用 `client.has_collection()` 检查集合是否存在
2. 如果不存在, 调用 `_create_collection()` 创建集合

3. 创建过程包括：构建 Schema → 构建索引参数 → 创建集合

幂等性保证：

- 只有在集合不存在时才创建
- 重复执行不会报错或创建重复集合

代码片段：

```
1  # 3. 创建集合（如不存在）
2  if not client.has_collection(collection_name=collection_name):
3      logger.info(f"step_2: 创建集合: {collection_name}")
4      _create_collection(client, collection_name, vector_dim)
```

Step4: 构建集合 Schema

定义集合的字段结构。

字段定义：

1. **chunk_id**: INT64 主键，自动生成
2. **content**: VARCHAR，存储切片文本内容
3. **title**: VARCHAR，存储切片标题
4. **parent_title**: VARCHAR，存储父级标题
5. **file_title**: VARCHAR，存储源文件标题
6. **sparse_vector**: SPARSE_FLOAT_VECTOR，存储稀疏向量
7. **dense_vector**: FLOAT_VECTOR，存储稠密向量

代码片段：

```
1  def _build_schema(client, vector_dim: int):
2      """构建集合 Schema。"""
3
4      # 1. 定义 schema
5      schema = client.create_schema(enable_dynamic_fields=True)
6
7      # 2. 添加主键(自增)
8      schema.add_field(
9          field_name="chunk_id",
10         datatype=DataType.INT64,
11         is_primary=True,
12         auto_id=True,
13     )
14
15     # 3. 添加标量字段
16     schema.add_field(field_name="content", datatype=DataType.VARCHAR,
17                       max_length=65535)
18     schema.add_field(field_name="title", datatype=DataType.VARCHAR,
19                       max_length=65535)
20     schema.add_field(field_name="parent_title", datatype=DataType.VARCHAR,
21                       max_length=65535)
22     schema.add_field(field_name="file_title", datatype=DataType.VARCHAR,
23                       max_length=65535)
24
25     # 4. 添加向量字段
26     schema.add_field(field_name="sparse_vector",
27                       datatype=DataType.SPARSE_FLOAT_VECTOR)
28     schema.add_field(field_name="dense_vector", datatype=DataType.FLOAT_VECTOR,
29                       dim=vector_dim)
```

```
24
25     # 5. 返回 schema
26     return schema
```

Step5: 构建索引参数

为稠密向量和稀疏向量分别配置索引。

索引配置：

1. 稠密向量索引：
 - 类型：AUTOINDEX（自动选择最优算法）
 - 度量：IP（内积）
2. 稀疏向量索引：
 - 类型：SPARSE_INVERTED_INDEX（倒排索引）
 - 度量：IP（内积）
 - 算法：DAAT_MAXSCORE（动态剪枝，高效查询）

代码片段：

```
1  def _build_index_params(client):
2      """构建索引参数。"""
3
4      # 1. 获取索引对象
5      index_params = client.prepare_index_params()
6
7      # 2. 稠密向量添加索引
8      index_params.add_index(
9          field_name="dense_vector",
10         index_name="dense_vector_index",
11         index_type="AUTOINDEX",
12         metric_type="IP",
13     )
14
15     # 3. 稀疏向量添加索引
16     index_params.add_index(
17         field_name="sparse_vector",
18         index_name="sparse_inverted_index",
19         index_type="SPARSE_INVERTED_INDEX",
20         metric_type="IP",
21         params={"inverted_index_algo": "DAAT_MAXSCORE"},
22     )
23
24     return index_params
```

Step6: 创建集合

组合 Schema 和索引参数，创建集合。

处理逻辑：

1. 调用 `_build_schema()` 构建 Schema
2. 调用 `_build_index_params()` 构建索引参数
3. 调用 `client.create_collection()` 创建集合

代码片段：

```
1  def _create_collection(client, collection_name: str, vector_dim: int):
```

```

2     """创建 Milvus 集合 (schema + 索引)。"""
3
4     # 1. 构建 schema
5     schema = _build_schema(client, vector_dim)
6
7     # 2. 构建索引
8     index_params = _build_index_params(client)
9
10    # 3. 创建集合
11    client.create_collection(
12        collection_name=collection_name,
13        schema=schema,
14        index_params=index_params,
15    )
16
17    logger.info(f"集合 {collection_name} 创建成功")

```

Step7: 批量插入数据

将切片数据批量插入到 Milvus 集合中。

输入：

- `chunks`: 包含向量的切片数据列表

处理逻辑：

1. 调用 `client.insert()` 批量插入数据
2. Milvus 会自动为数据生成 `chunk_id`
3. 插入结果包含 `insert_count` 和 `ids`

注意事项：

- 插入时不需要提供 `chunk_id` 字段 (`auto_id=True`)
- Milvus 会自动处理数据类型转换

代码片段：

```

1     def _insert_and_backfill_ids(client, collection_name: str, chunks: List[Dict]):
2         """插入数据并将 Milvus 返回的主键回填到每条 chunk。"""
3
4         result = client.insert(collection_name=collection_name, data=chunks)
5
6         insert_count = result.get("insert_count", 0)
7         logger.info(f"成功插入 {insert_count} 条数据")

```

Step8: 回填主键到业务数据

将 Milvus 返回的主键回填到原始切片数据中。

处理逻辑：

1. 从插入结果中获取 `ids` 列表
2. 验证返回的 ID 数量与切片数量一致
3. 使用 `zip()` 将 ID 逐一回填到对应的 chunk
4. 将 ID 转换为字符串存储 (便于后续 JSON 序列化和跨系统传递)

代码片段：

```

1  # 回填 chunk_id
2  inserted_ids = result.get("ids", [])
3  if inserted_ids and len(inserted_ids) == len(chunks):
4      for chunk, chunk_id in zip(chunks, inserted_ids):
5          chunk["chunk_id"] = str(chunk_id)
6  else:
7      logger.warning(
8          f"回填 chunk_id 失败: 返回 {len(inserted_ids)} 个 ID, "
9          f"期望 {len(chunks)} 个"
10     )

```

2.4.2 具体实现

```

1  import json
2  import logging
3  import os
4  from typing import List, Dict
5  from milvus_util import get_milvus_client
6  from pymilvus import DataType
7
8  logging.basicConfig(
9      level=logging.DEBUG,
10     format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
11 )
12 logger = logging.getLogger(__name__)
13
14 """
15 Milvus 导入
16 将向量化后的切片数据批量导入 Milvus, 自动创建集合和索引（如果不存在）。
17 代码结构:
18     1. 主流程 (process)
19     2. 集合创建 (schema 构建 + 索引构建 + 创建集合)
20     3. 数据插入与 chunk_id 回填
21     4. 测试入口
22 """
23
24 chunks_collection="kb_chunks_v2"
25 def process(chunks):
26     # 1. 校验
27     if not chunks:
28         logger.warning("chunks 为空, 跳过导入")
29         return None
30
31     vector_dim = _get_vector_dim(chunks)
32     logger.info(f"step_1:准备导入 {len(chunks)} 条数据, 向量维度: {vector_dim}")
33
34     # 2. 连接 & 操作
35     try:
36         client = get_milvus_client()
37         collection_name = chunks_collection
38
39         # 3. 创建集合 (如不存在)
40         if not client.has_collection(collection_name=collection_name):
41             logger.info(f"step_2: 创建集合: {collection_name}")
42             _create_collection(client, collection_name, vector_dim)
43
44         # 4. 插入数据, 并更新 chunks

```

```

45         logger.info("step_3: 执行插入")
46         _insert_and_backfill_ids(client, collection_name, chunks)
47     except Exception as e:
48         raise Exception(f"Milvus 操作失败: {e}")
49     # 5. 返回更新后的 chunks 状态
50     return chunks
51
52 def _get_vector_dim(chunks: List[Dict]) -> int:
53     """从首条 chunk 中获取稠密向量维度。
54     Args:
55         chunks: 切片数据列表。
56     Returns:
57         向量维度。
58     Raises:
59         MilvusError: 首条 chunk 不包含 dense_vector 时抛出。
60     """
61     dim = len(chunks[0].get("dense_vector", []))
62     if dim == 0:
63         raise Exception(f"切片数据不包含 dense_vector")
64     return dim
65
66 # ===== #
67 #                               2. 集合创建                               #
68 # ===== #
69 def _create_collection( client, collection_name: str, vector_dim: int):
70     """创建 Milvus 集合 (schema + 索引)。
71     Args:
72         client: MilvusClient 实例。
73         collection_name: 集合名称。
74         vector_dim: 稠密向量维度。
75     """
76     # 1. 构建 schema
77     schema = _build_schema(client, vector_dim)
78
79     # 2. 构建索引
80     index_params = _build_index_params(client)
81
82     # 3. 创建集合
83     client.create_collection(
84         collection_name=collection_name,
85         schema=schema,
86         index_params=index_params,
87     )
88
89     logger.info(f"集合 {collection_name} 创建成功")
90
91 def _build_schema(client, vector_dim: int):
92     """构建集合 Schema。
93     Args:
94         client: MilvusClient 实例。
95         vector_dim: 稠密向量维度。
96     Returns:
97         CollectionSchema 对象。
98     """
99
100     # 1. 定义 schema
101     schema = client.create_schema(enable_dynamic_fields=True)
102

```

```

103     # 2. 添加主键(自增)
104     schema.add_field(
105         field_name="chunk_id",
106         datatype=DataType.INT64,
107         is_primary=True,
108         auto_id=True,
109     )
110
111     # 3. 添加标量字段
112     schema.add_field(field_name="content", datatype=DataType.VARCHAR,
113 max_length=65535)
114     schema.add_field(field_name="title", datatype=DataType.VARCHAR,
115 max_length=65535)
116     schema.add_field(field_name="parent_title", datatype=DataType.VARCHAR,
117 max_length=65535)
118     schema.add_field(field_name="file_title", datatype=DataType.VARCHAR,
119 max_length=65535)
120
121     # 4. 添加向量字段
122     schema.add_field(field_name="sparse_vector",
123 datatype=DataType.SPARSE_FLOAT_VECTOR)
124     schema.add_field(field_name="dense_vector", datatype=DataType.FLOAT_VECTOR,
125 dim=vector_dim)
126
127     # 5. 返回 schema
128     return schema
129
130 def _build_index_params(client):
131     """构建索引参数。
132     Args:
133         client: MilvusClient 实例。
134     Returns:
135         IndexParams 对象。
136     """
137
138     # 1. 获取索引对象
139     index_params = client.prepare_index_params()
140
141     # 2. 稠密向量添加索引
142     index_params.add_index(
143         field_name="dense_vector",
144         index_name="dense_vector_index",
145         index_type="AUTOINDEX",
146         metric_type="IP",
147     )
148
149     # 3. 稀疏向量添加索引
150     index_params.add_index(
151         field_name="sparse_vector",
152         index_name="sparse_inverted_index",
153         index_type="SPARSE_INVERTED_INDEX",
154         metric_type="IP",
155         params={"inverted_index_algo": "DAAT_MAXSCORE"},
156     )
157     return index_params
158
159 def _insert_and_backfill_ids(client, collection_name: str, chunks: List[Dict],):
160     """插入数据并将 Milvus 返回的主键回填到每条 chunk。

```

```

155     Args:
156         client: MilvusClient 实例。
157         collection_name: 目标集合名称。
158         chunks: 切片数据列表（会被原地修改，添加 chunk_id 字段）。
159     """
160     result = client.insert(collection_name=collection_name, data=chunks)
161
162     insert_count = result.get("insert_count", 0)
163     logger.info(f"成功插入 {insert_count} 条数据")
164
165     # 回填 chunk_id
166     inserted_ids = result.get("ids", [])
167     if inserted_ids and len(inserted_ids) == len(chunks):
168         for chunk, chunk_id in zip(chunks, inserted_ids):
169             chunk["chunk_id"] = str(chunk_id)
170     else:
171         logger.warning(
172             f"回填 chunk_id 失败: 返回 {len(inserted_ids)} 个 ID, "
173             f"期望 {len(chunks)} 个"
174         )
175
176 if __name__ == "__main__":
177     """
178     测试流程:
179     1. 从上一个节点的输出文件读取状态（带向量的切片数据）
180     2. 执行 Milvus 导入
181     3. 验证回填的 chunk_id
182     4. 将结果保存到临时文件
183     """
184     # -----
185     # Step 1: 配置路径
186     # -----
187     temp_dir = r".\result\sample\auto"
188
189     # 输入: 上一个向量化节点处理后的状态
190     input_path = os.path.join(temp_dir, "chunks_1_vector.json")
191
192     # 输出: 导入 Milvus 后的状态 (含 chunk_id)
193     output_path = os.path.join(temp_dir, "chunks_1_vector_ids.json")
194
195     # -----
196     # Step 2: 读取输入数据
197     # -----
198     print(f"正在读取输入文件: {input_path}")
199
200     with open(input_path, "r", encoding="utf-8") as f:
201         chunks = json.load(f)
202
203     print(f"读取到 {len(chunks)} 个切片")
204
205     # -----
206     # Step 3: 构建状态并执行处理
207     # -----
208
209     print("\n开始执行 Milvus 导入...")
210     output_chunks = process(chunks)
211
212     # -----

```

```

213     # Step 4: 验证回填结果
214     # -----
215     print(f"\n处理完成, 共 {len(output_chunks)} 个切片")
216
217     # 检查 chunk_id 回填情况
218     chunks_with_id = sum(1 for c in output_chunks if c.get("chunk_id"))
219     chunks_without_id = len(output_chunks) - chunks_with_id
220
221     print(f"\n回填统计:")
222     print(f"    - 成功回填 chunk_id: {chunks_with_id} 个")
223     print(f"    - 未回填 chunk_id: {chunks_without_id} 个")
224
225     # 打印前 3 个切片的信息
226     print("\n前 3 个切片信息:")
227     for i, chunk in enumerate(output_chunks[:3]):
228         print(f"\n    切片 {i + 1}:")
229         print(f"        chunk_id: {chunk.get('chunk_id', '无')}")
230         print(f"        title: {chunk.get('title', '')}")
231         print(f"        file_title: {chunk.get('file_title', '')}")
232         print(f"        content: {chunk.get('content', '')[:50]}...")
233
234     # -----
235     # Step 5: 保存输出文件
236     # -----
237     with open(output_path, "w", encoding="utf-8") as f:
238         json.dump(output_chunks, f, ensure_ascii=False, indent=4)
239
240     print(f"\n已保存到: {output_path}")

```

执行后输出结果如下:

```

1  (rag) D:\LLM\backup\03-rag>python import_milvus.py
2  INFO:numexpr.utils:NumExpr defaulting to 16 threads.
3  正在读取输入文件: D:\LLM\backup\03-rag\result\sample\auto\chunks_1_vector.json
4  读取到 26 个切片
5
6  开始执行 Milvus 导入...
7  INFO:mineru:step_1:准备导入 26 条数据, 向量维度: 1024
8  INFO:mineru:step_2: 创建集合: kb_chunks_v2
9  INFO:mineru:集合 kb_chunks_v2 创建成功
10 INFO:mineru:step_3: 执行插入
11 INFO:mineru:成功插入 26 条数据
12
13 处理完成, 共 26 个切片
14
15 回填统计:
16     - 成功回填 chunk_id: 26 个
17     - 未回填 chunk_id: 0 个
18
19 前 3 个切片信息:
20
21     切片 1:
22         chunk_id: 468309799367345842
23         title:
24         file_title:
25         content: ![中国科学院国家天文台 (NAOC) 标志](http://127.0.0.1:9000/knowle...
26
27     切片 2:

```

```
28     chunk_id: 468309799367345843
29     title: # 中国科学院国家天文台2023 年部门预算
30     file_title:
31     content: # 中国科学院国家天文台2023 年部门预算
32
33     ![中国科学院国家天文台园区外观](http://1...
34
35     切片 3:
36     chunk_id: 468309799367345844
37     title: ## 一、中国科学院国家天文台基本情况
38     file_title:
39     content: ## 一、中国科学院国家天文台基本情况
40
41     中国科学院国家天文台是中国科学院直属事业单位，成立于 20...
42
43     已保存到: D:\LLM\backup\03-rag\result\sample\auto\chunks_1_vector_ids.json
```